SwiftDrop – Parcel Delivery & Courier Management System

Design Report · Group project · 01 November 2026

Repository: https://github.com/Sachintha-dilshan49/SwiftDrop (design state: branch `docs/report-nov1`, based on `dev` at commit `2c99307`)

#	Role	Member	Owns
1	Team lead, architecture, gateway, pricing	N.L. Sachintha Dilshan	`api-gateway`, `pricing-service`, `common`, `docs`
2	Authentication and user management	Thulani Jayathilake	`auth-service`
3	Parcel and live tracking	Sanduni Liyanage	`parcel-service`
4	Dispatch and courier	Tharuki	`dispatch-service`
5	Merchant platform	Sudari Ratnayake	`merchant-service`
6	Events, notifications and webhooks	Thisaru Ramanayaka	`notification-service`, RabbitMQ topology
7	Frontend and real-time UI	Supuni	`frontend`
8	DevOps: Docker, Kubernetes, CI/CD	Kevinda	`deploy`, `.github/workflows`
9	AI assistant and testing coordinator	Devmith	`ai-assistant`, test setup, coverage

Contents

1. Introduction and scope
2. Requirements
3. Use case diagrams
4. Architecture
5. Integration design
6. Data design
7. Key flows
8. Security design
9. Pricing design
10. Design patterns and principles
11. Technology choices
12. UI design
13. Current implementation status
14. Testing strategy and coverage
15. Team contributions
16. Appendix: decisions and glossary

1. Introduction and scope

SwiftDrop is a parcel delivery and courier management platform for Colombo and nearby cities. Customers book and track deliveries, merchants send orders from their online stores through an API, couriers update deliveries from their phones, and dispatchers and admins run daily operations. Anyone with a tracking number can follow a parcel live without logging in.

The system is a set of Spring Boot microservices behind a Spring Cloud Gateway, with a React single-page app, PostgreSQL (one database per service), RabbitMQ for status events, Redis for rate limits and MinIO for delivery photos. It runs with one Docker Compose command; a Helm chart for Kubernetes is the bonus target.

In scope: four user types plus guests with role-based access; JWT authentication and Google login (OAuth 2.0); booking with a server-side fee; a strict parcel status lifecycle; real-time tracking with Server-Sent Events; asynchronous status events that trigger email, SMS and signed merchant webhooks with retry; dispatcher assignment and daily reports; an AI assistant with tools that call our own APIs; containerised deployment and CI.

Out of scope: online payment, route optimisation, real SMS gateways (SMS is mocked) and native mobile apps (the SPA is mobile-first).

This report describes the design that the team agreed and records the implementation state on 01 November 2026. `docs/architecture.md` v1.2 in the repository is the source of truth; every diagram in this report was checked against it.

2. Requirements

2.1 Actors

Actor	Description
Guest	Anyone without an account. Tracks parcels, gets a fee estimate, signs up.
Customer	Registered user (`CUSTOMER`). Books, tracks and pays for deliveries.
Merchant	Business user (`MERCHANT`), approved by an admin. Uses the dashboard and the order API.
Merchant store server	External system that calls the merchant API with an API key and receives webhooks.
Courier	Delivery person (`COURIER`) with a phone. Never called "rider" or "driver".
Dispatcher	Operations staff (`DISPATCHER`). Assigns parcels, watches workload and reports.
Admin	`ADMIN`. Everything a dispatcher can do, plus users, merchants, zones and pricing.
Recipient	The person receiving a parcel. Has no account; receives SMS (and email for merchant orders).

2.2 Functional requirements

ID	Actor	Requirement
FR-G1	Guest	Track a parcel by tracking number and see live status updates without logging in.
FR-G2	Guest	Get a fee estimate for a route, weight, size, speed and promo code.
FR-G3	Guest	Sign up as a customer with email or mobile number and password, or with Google.
FR-G4	Guest	Register a merchant business; it starts as `PENDING`.
FR-C1	Customer	Book a delivery with pickup and drop-off address and zone, recipient, size, weight, speed and optional promo code; see the fee before confirming.
FR-C2	Customer	Receive a tracking number such as `SD-7K3Q-9P2M` and an email on every status change.
FR-C3	Customer	List own deliveries, see details and status history, download a PDF receipt.
FR-C4	Customer	Ask the AI assistant where a parcel is, how much a delivery costs, or for a list of own parcels.

ID	Actor	Requirement
FR-M1	Merchant	View approval status; once approved, create and revoke API keys (shown once).
FR-M2	Merchant	Configure a webhook URL and signing secret.
FR-M3	Store server	Create orders through `POST /api/v1/merchant-api/orders` with `X-API-Key`; repeated requests with the same idempotency key do not create duplicates.
FR-M4	Store server	Receive an HMAC-signed webhook on every status change, retried on failure.
FR-K1	Courier	See today's assigned deliveries.
FR-K2	Courier	Update a delivery's status with a note and an optional proof photo; mark it `FAILED` with a reason.
FR-D1	Dispatcher	See unassigned parcels and courier workload; assign parcels to couriers.
FR-D2	Dispatcher	See daily reports; list webhook deliveries and replay parked ones.
FR-A1	Admin	Approve or reject merchants with a reason.
FR-A2	Admin	Manage users and roles; manage zones, pricing rules and promo codes.
FR-S1	System	Allow only `CREATED → PICKED_UP → IN_TRANSIT → OUT_FOR_DELIVERY → DELIVERED
FR-S2	System	Calculate every binding fee in pricing-service; never trust a fee sent by a client.

2.3 Non-functional requirements

Quality	Requirement and how the design meets it
Security	BCrypt passwords; RS256 JWTs verified with JWKS; role and ownership checks in every service; API keys stored hashed; `/internal/**` protected by a shared token (ADR-010).
Privacy	No personal data in logs, errors or events; log masking in `common`; public tracking shows status only.
Reliability	Transactional outbox so no status event is lost; idempotent consumers; webhook retry 10 s → 1 min → 5 min, then parking.
Responsiveness	Service-to-service calls time out after 2 s connect and 5 s read; live updates arrive by SSE without polling.
Availability	A failure in notification or dispatch does not block bookings, because they are decoupled by the broker.
Maintainability	One service per owner, layered code, shared `common` library, at least 70% line coverage per module enforced in CI.
Portability	Everything runs from Docker Compose; configuration only from environment variables.
Usability	Mobile-first SPA; the courier and public tracking screens are designed for phones first.

3. Use case diagrams

The overview shows every actor and its main use cases; the per-actor diagrams add the detail. «include» marks a step that always happens; «extend» marks an optional one with its condition.

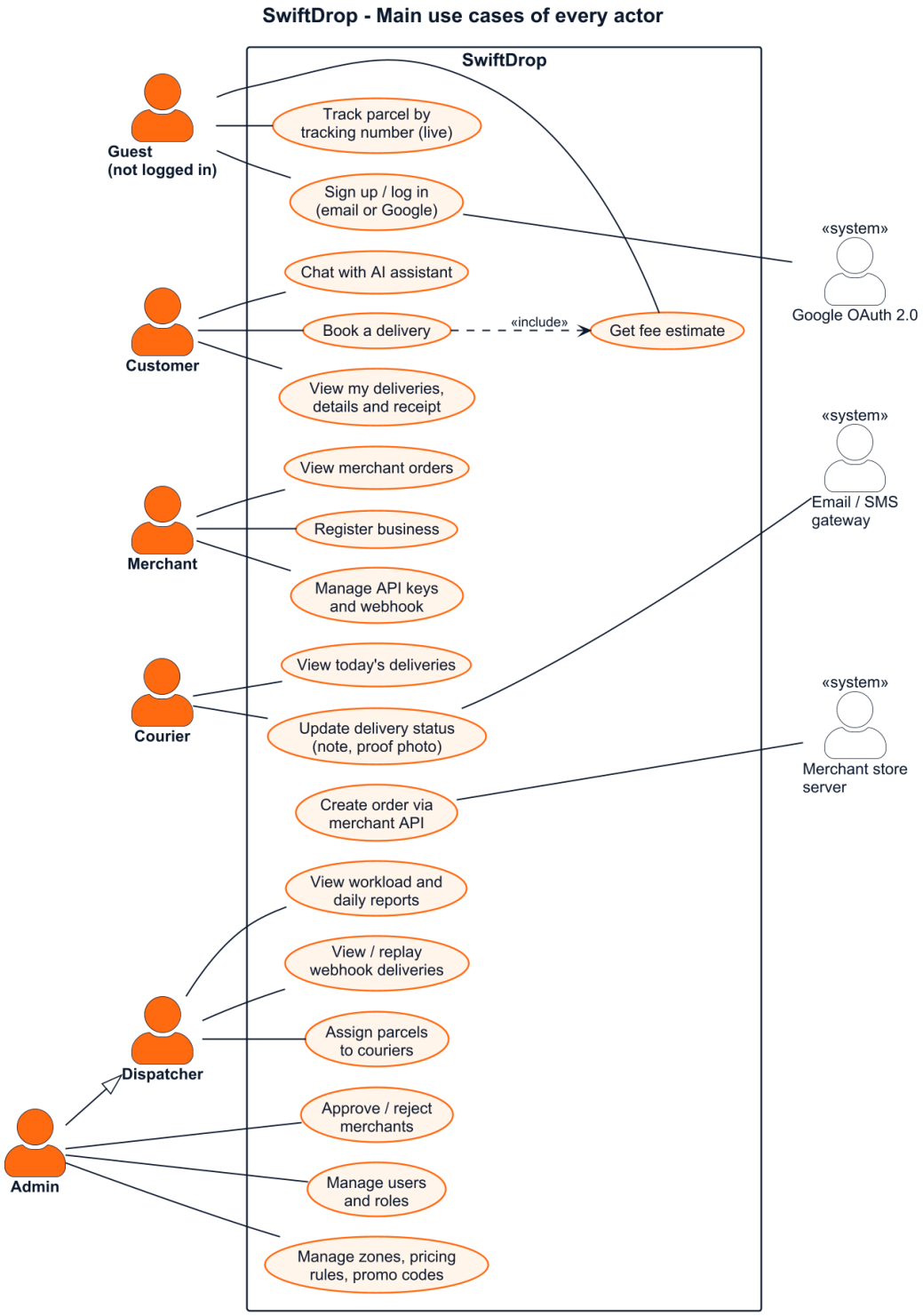


Figure 1. All actors and their main use cases

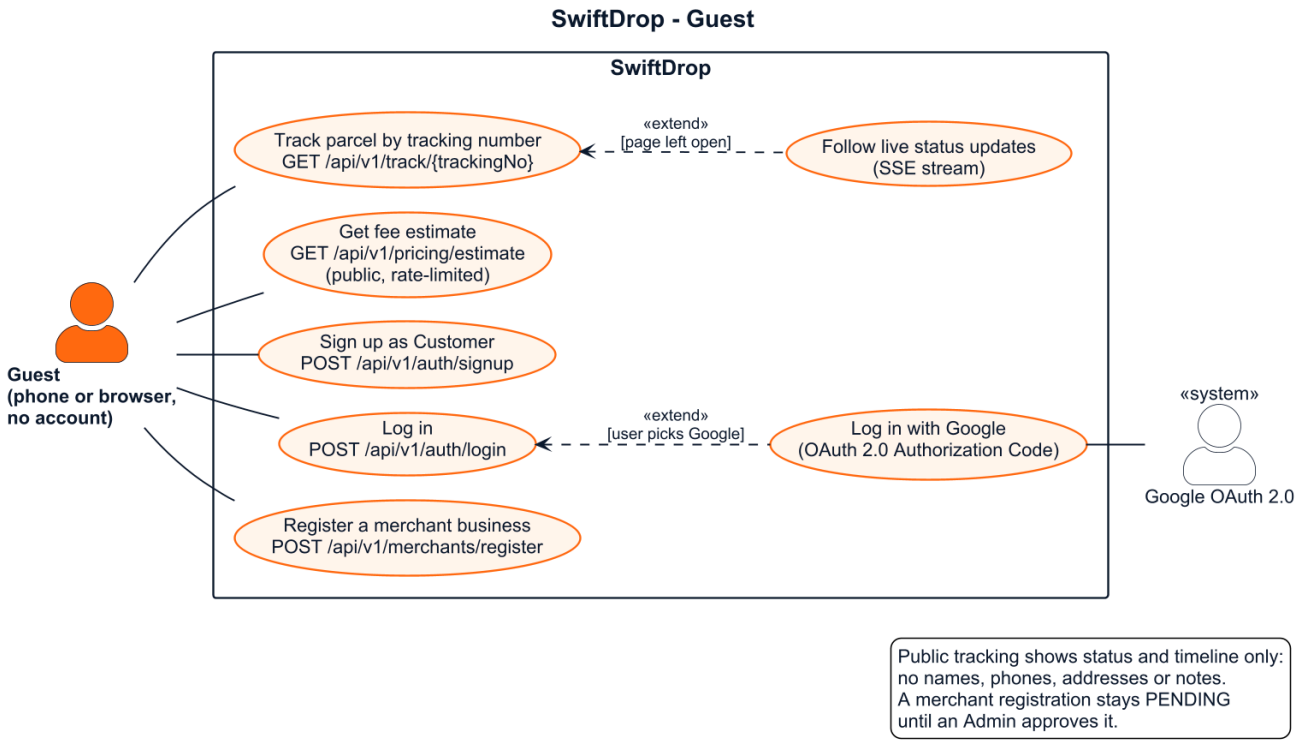


Figure 2. Guest: public tracking, fee estimate, sign-up and merchant registration

Guest (Figure 2). Tracking and the fee estimate need no account. The live stream extends tracking while the page stays open. Google login extends normal login.

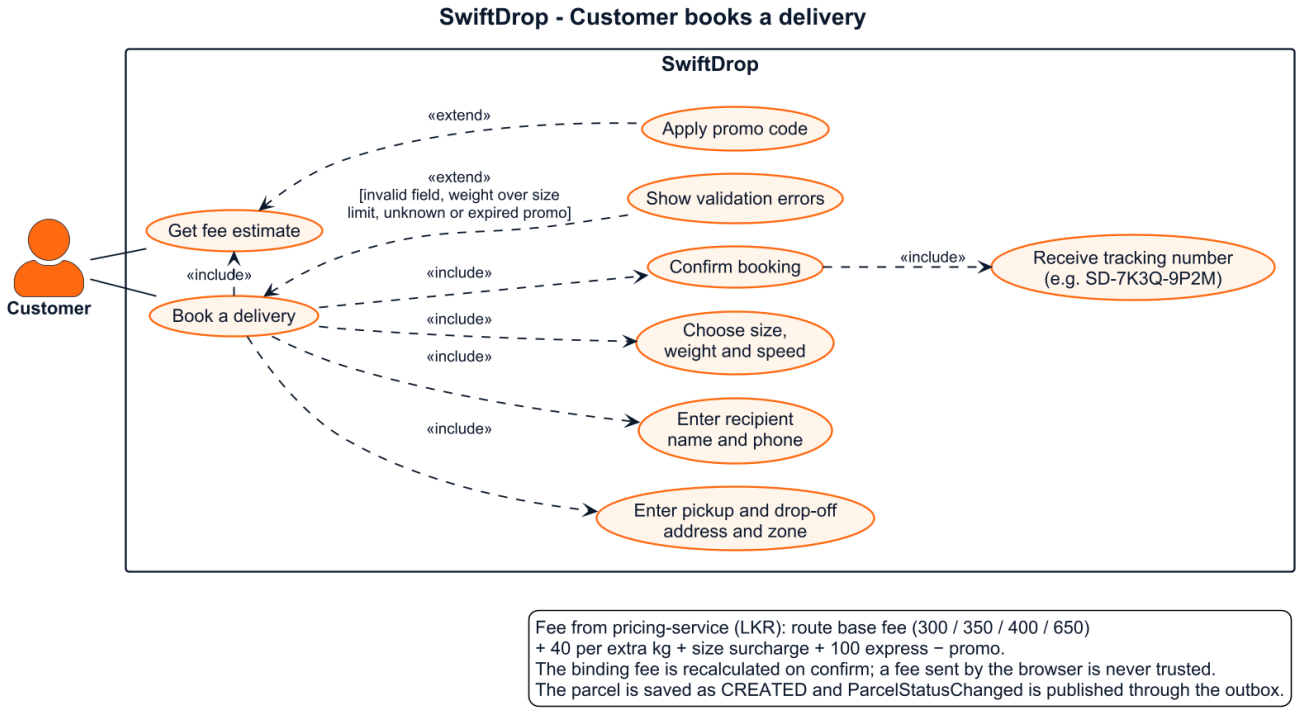


Figure 3. Customer books a delivery

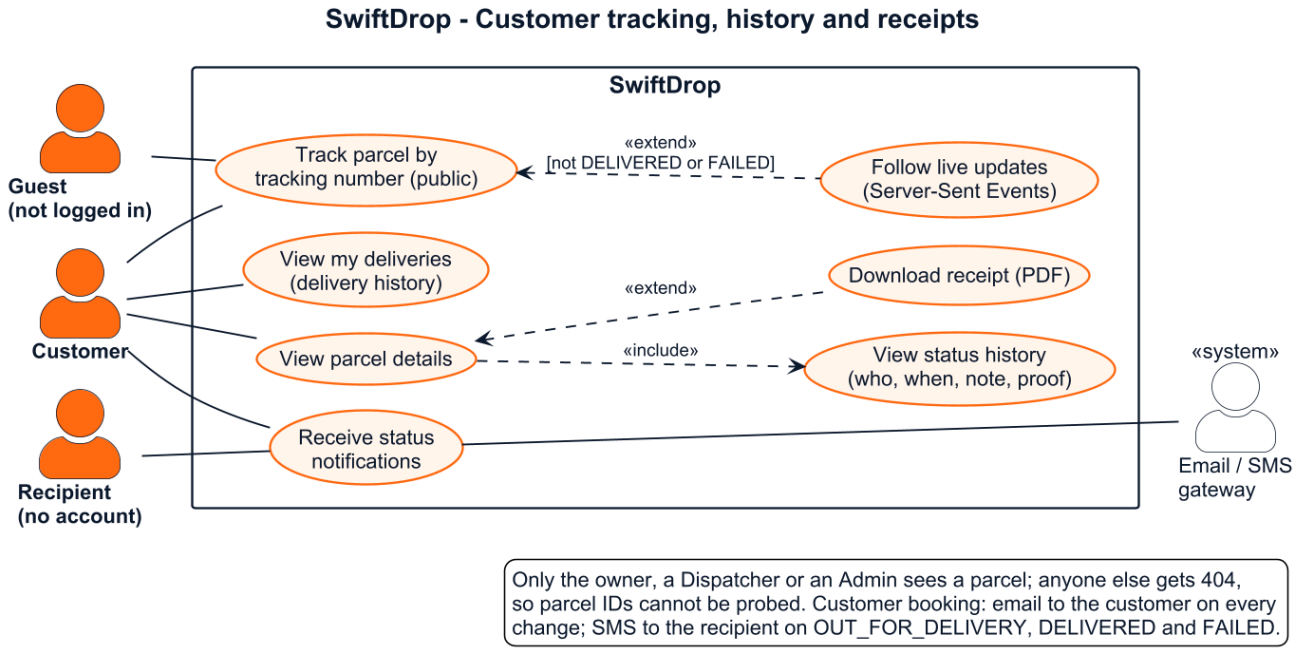


Figure 4. Customer tracking, history, receipts and notifications

Customer (Figures 3–4). Booking always includes the address, recipient, parcel and fee-estimate steps. Only the owner, a dispatcher or an admin can see a parcel; anyone else gets `404`, so parcel IDs cannot be probed. Both figures were recreated from Supuni's originals (Role 7) to match the pricing model and notification rules. Supuni's other customer diagrams are in the repository: the overview `UC-00_customer_overview`, account and sign-in `UC-01_customer_account`, and the AI assistant `UC-04_customer_ai_assistant`.

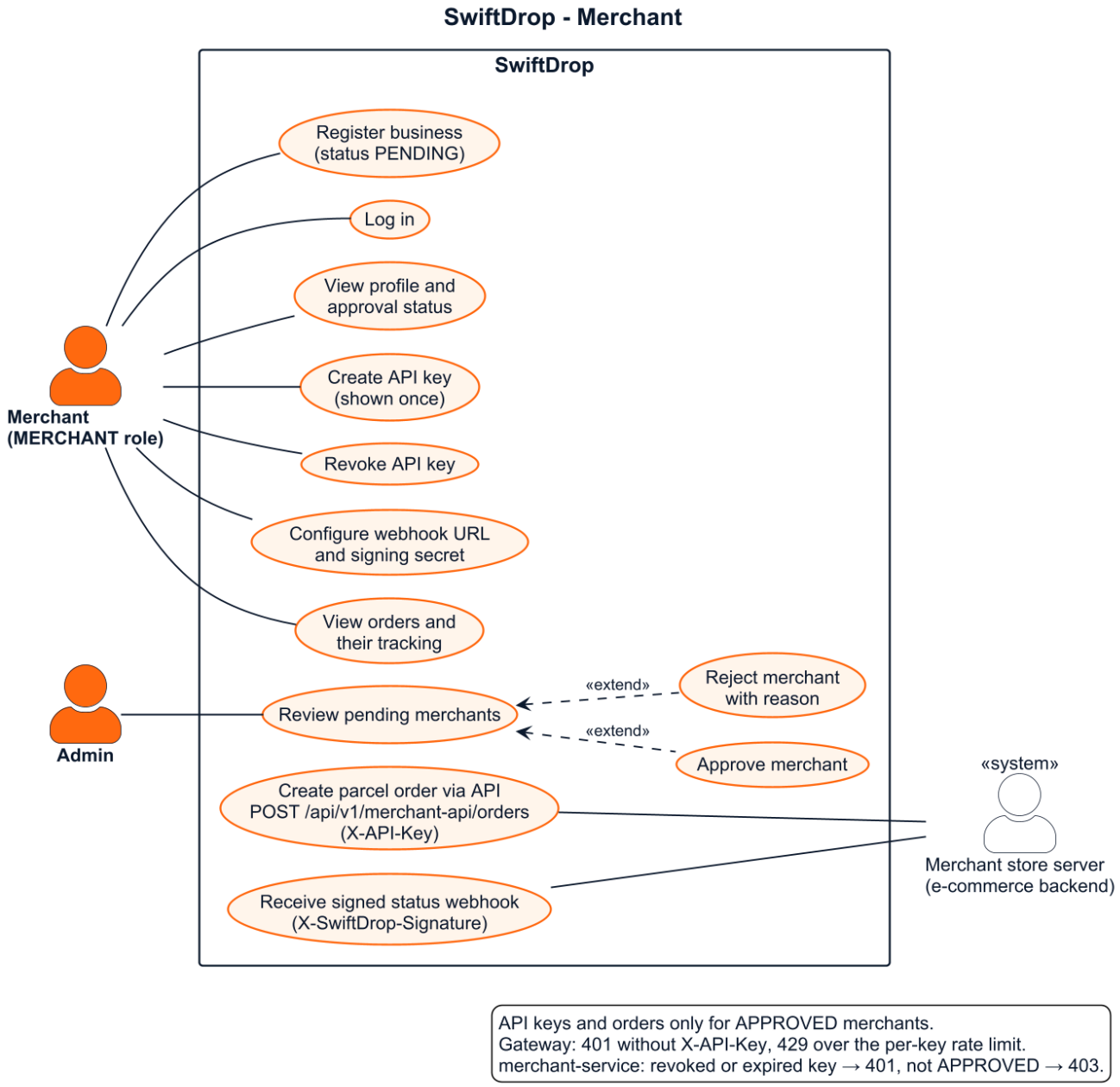


Figure 5. Merchant, merchant store server and admin approval

Merchant (Figure 5). Registration creates a `PENDING` merchant. Only an `APPROVED` merchant can create API keys and orders. The store server is a separate actor because it authenticates with an API key, not a user login.

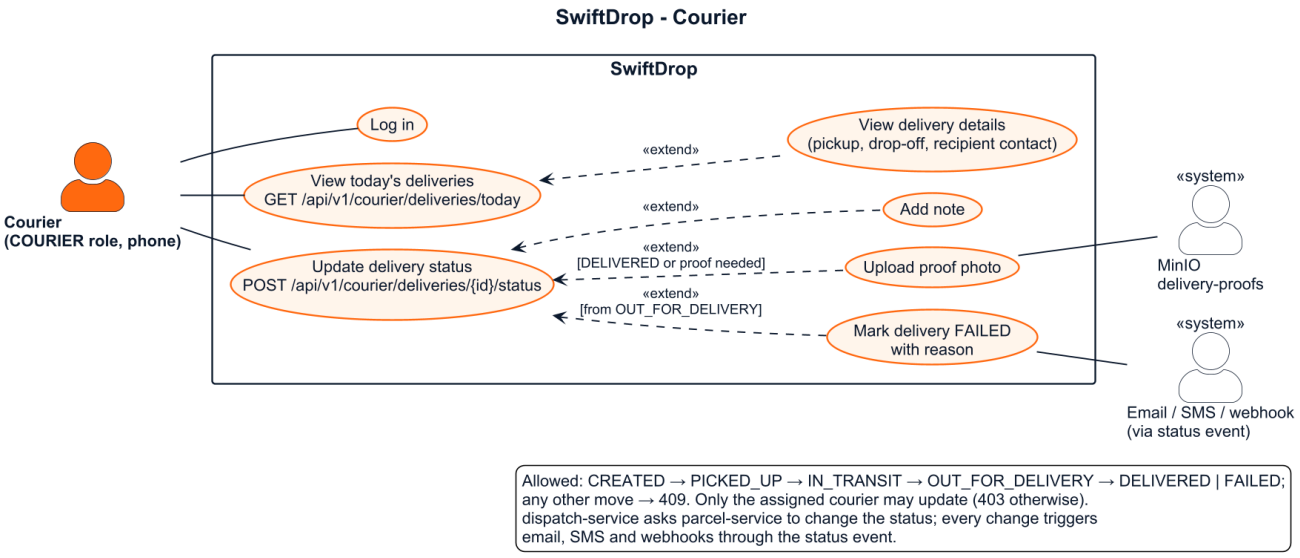


Figure 6. Courier

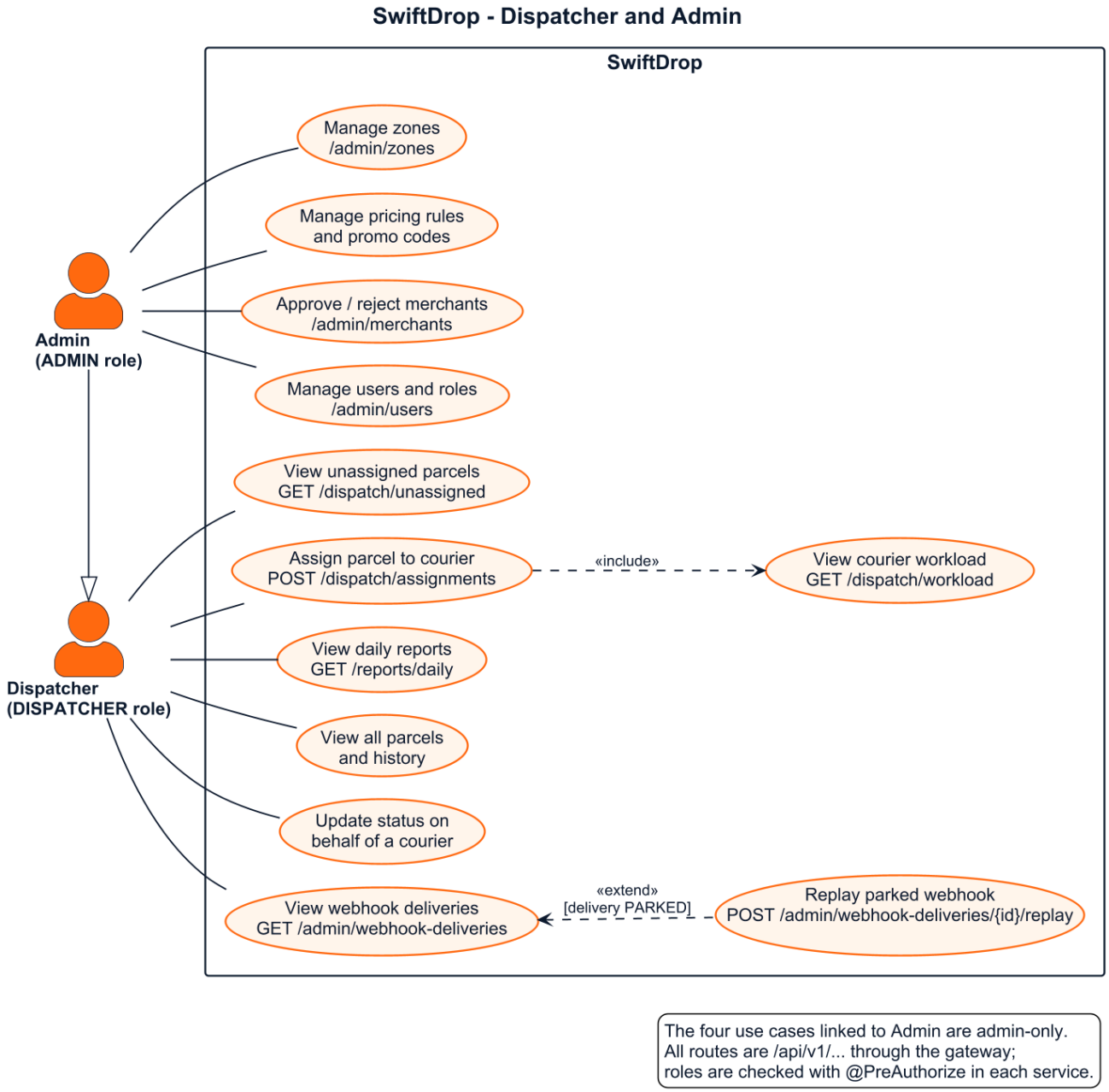


Figure 7. Dispatcher and Admin

Courier, Dispatcher and Admin (Figures 6–7). A courier can only update deliveries assigned to them. Admin generalises Dispatcher, so the admin-only use cases (merchants, users, zones and pricing) sit on top of the dispatcher's operational ones.

4. Architecture

4.1 Overview and layers

SwiftDrop is a set of **microservices behind an API gateway**. External traffic enters only through the gateway; services call each other with REST when they need an answer, and publish events to RabbitMQ when something happened.

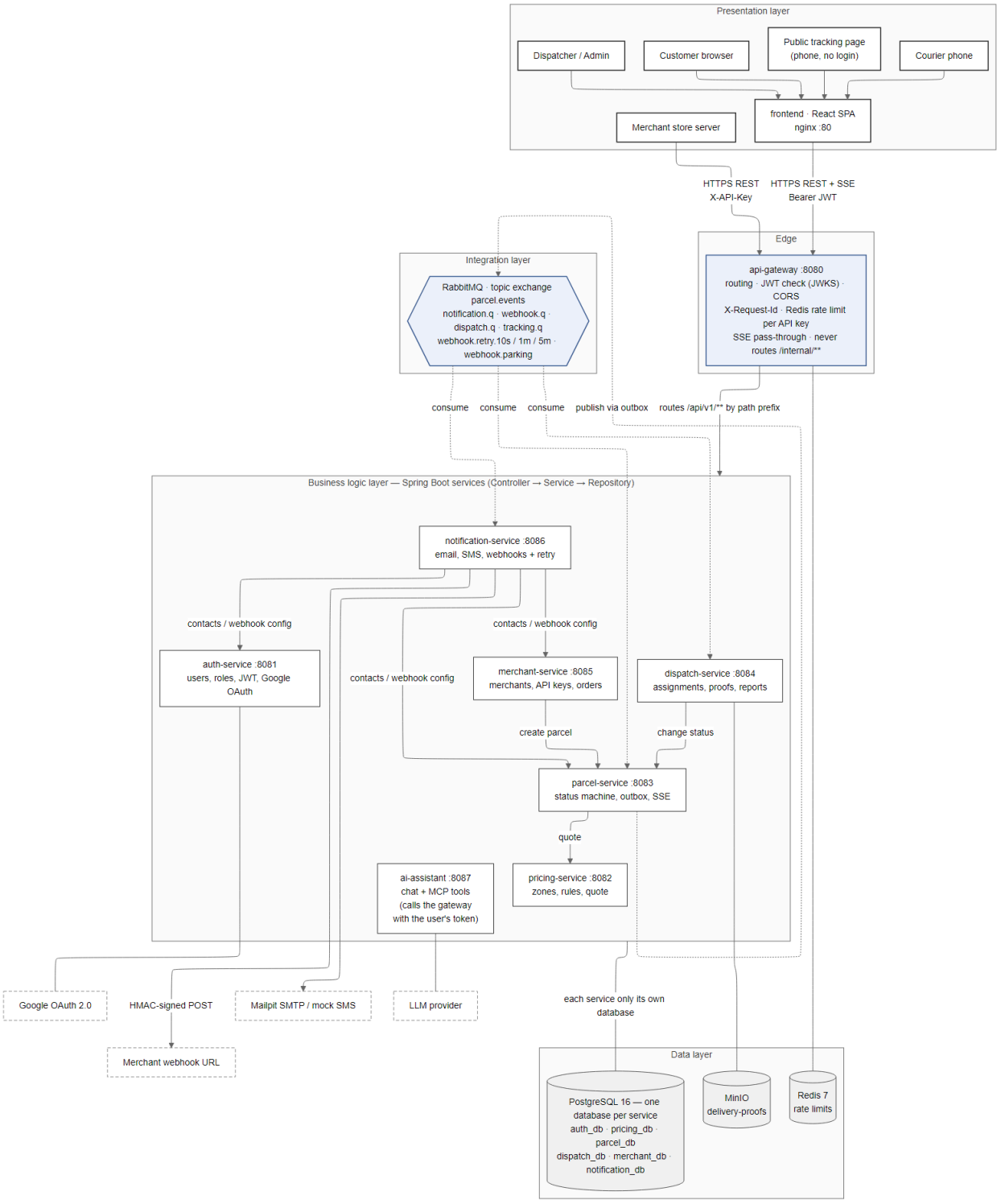


Figure 8. System by layer: presentation, edge, business logic, integration and data

Layer	Components	Responsibility
Presentation	React SPA (all roles), public tracking page	UI, client-side validation with zod, API clients, `EventSource` for SSE
Edge	ingress (Kubernetes), `api-gateway`	Single entry point: routing, JWT check, CORS, request IDs, rate limits
Business logic	Seven Spring Boot services	Rules, authorization and ownership, each layered Controller → Service → Repository
Integration	RabbitMQ exchange and queues	Asynchronous status events, webhook retry

Layer	Components	Responsibility
Data	PostgreSQL (one database per service), Redis, MinIO	Persistence, rate-limit counters, proof photos

Inside each service the same three layers repeat (Figure 9). Controllers validate requests and check roles, services hold the business rules and transactions, and repositories are the only code that touches the database. The shared `common` library adds the security filter chains, the RFC 7807 error handler and log masking to every service automatically.

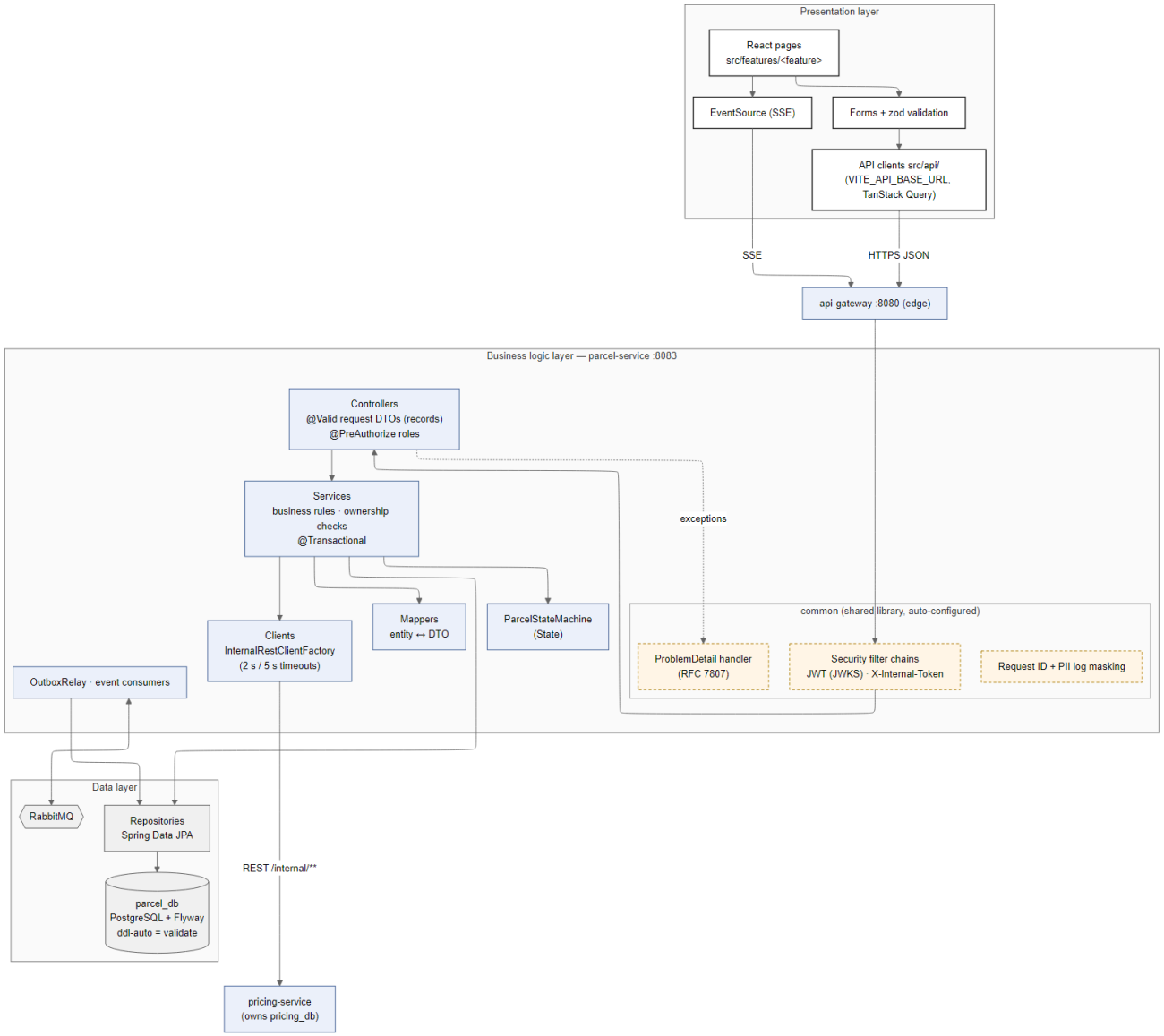


Figure 9. Presentation, business logic and data layers inside one request (parcel-service)

4.2 Services

Service	Port	Owner	Database	Responsibility
`api-gateway`	8080	Role 1	— (Redis)	Entry point, JWT verification, routing, CORS, rate limits, request IDs
`auth-service`	8081	Role 2	`auth_db`	Sign-up, login, JWT, refresh tokens, Google OAuth 2.0, roles
`pricing-service`	8082	Role 1	`pricing_db`	Zones, pricing rules, promo codes, fee estimate and quote
`parcel-service`	8083	Role 3	`parcel_db`	Parcels, status machine, history, receipt, SSE, event publishing

Service	Port	Owner	Database	Responsibility
`dispatch-service`	8084	Role 4	`dispatch_db`	Couriers, assignments, day list, proof photos, reports
`merchant-service`	8085	Role 5	`merchant_db`	Merchants, approval, API keys, order API, webhook settings
`notification-service`	8086	Role 6	`notification_db`	Event consumers, email, mock SMS, webhooks with retry
`ai-assistant`	8087	Role 9	—	Chat endpoint and MCP tools that call our APIs
`frontend`	80	Role 7	—	React SPA for all roles

4.3 Deployment view

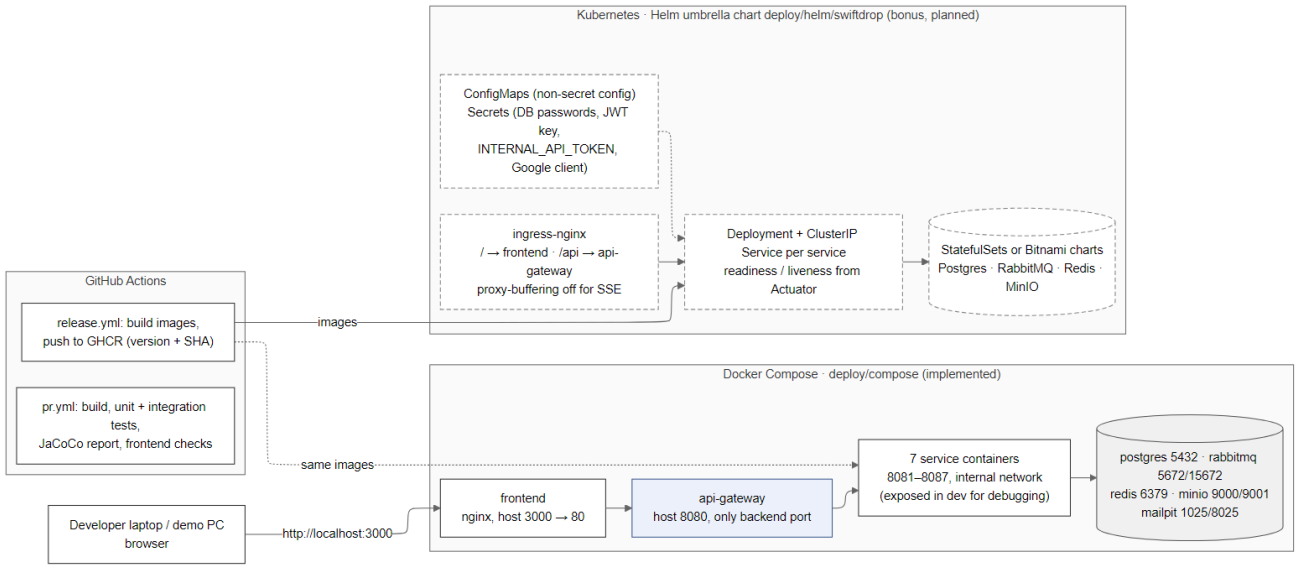


Figure 10. Deployment: Docker Compose (implemented) and Kubernetes with Helm (planned)

Locally, `docker compose up --build` starts the infrastructure, the seven services, the gateway and the frontend; only the frontend (3000) and the gateway (8080) need to be reachable from the browser. In Kubernetes, ingress-nginx sends `/` to the frontend and `/api` to the gateway, with response buffering switched off so SSE works. Each service is a Deployment with a ClusterIP Service and Actuator health probes; secrets come from Kubernetes Secrets. GitHub Actions builds and tests every pull request and pushes images to GHCR on release.

4.4 Justification

Microservices instead of a modular monolith (ADR-001). The brief allows a layered monolith, a modular monolith or microservices. A modular monolith would be simpler to run and debug. We chose microservices because each of the nine members owns, tests and demonstrates an independent service; the brief's integration patterns (publish/subscribe, broker, webhooks) then cross real network boundaries instead of in-process calls; and the Kubernetes bonus deploys several independent workloads. To keep the cost manageable we skip a service registry and a config server (Compose and Kubernetes DNS resolve names; configuration comes from environment variables) and run one PostgreSQL server with a separate database per service. The team reconfirmed this on 25 October.

Spring Boot 4.1 and Spring Cloud 2025.1 (ADR-009). One supported, compatible set for the gateway, security, data, messaging and Spring AI, supported until mid-2027, so no upgrade is needed during the project. Java 21 is the target.

Database per service (ADR-008). Each service owns its schema and Flyway migrations; there are no foreign keys or queries across databases. A shared database was considered and rejected twice, because it couples services and invites cross-service joins. Consistency comes from shared conventions and the team lead's review of every migration.

Custom auth-service instead of Keycloak (ADR-006). The auth owner has real, testable code to explain, and local setup stays light.

5. Integration design

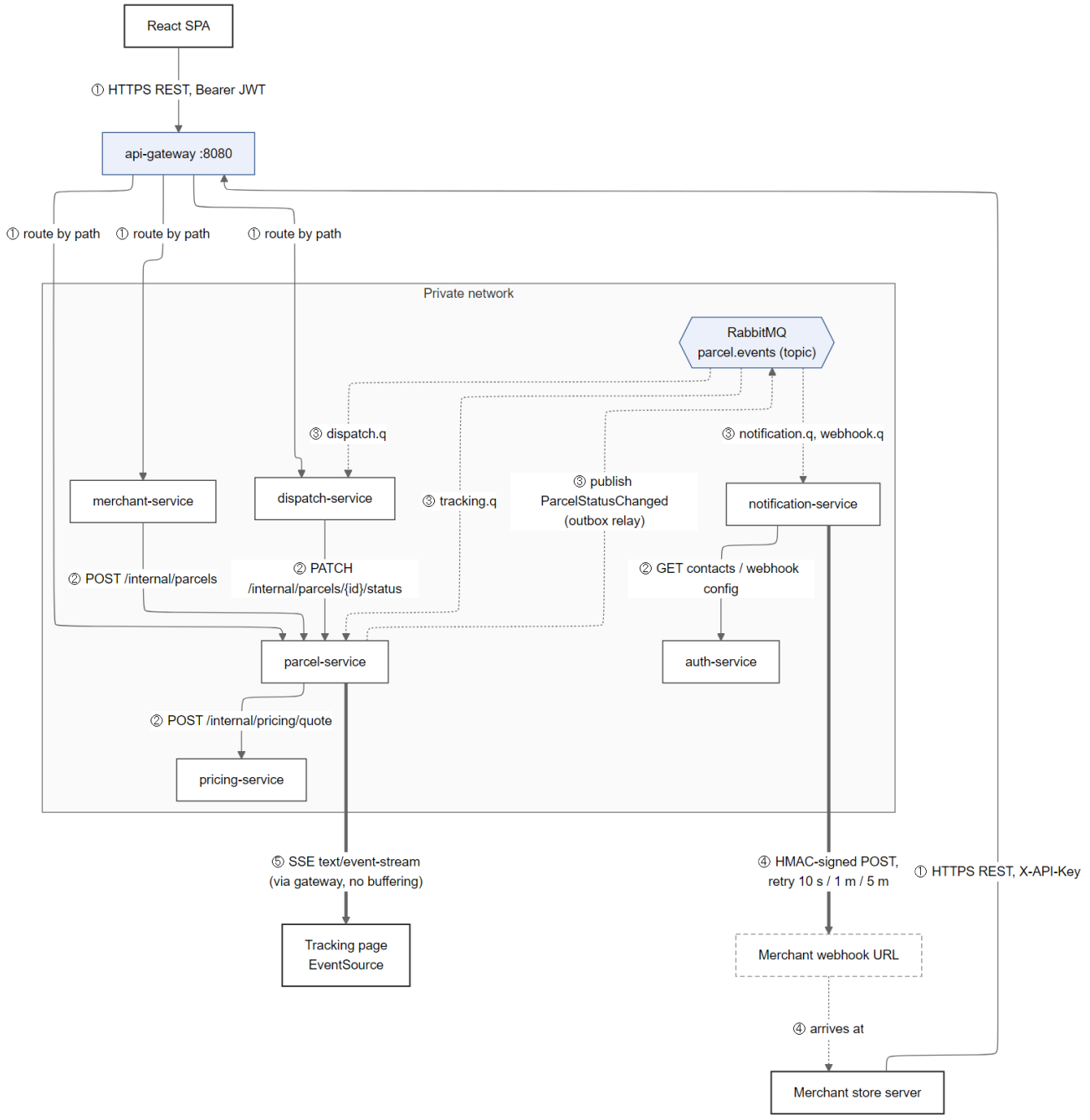


Figure 11. The five communication styles and where they are used

#	Pattern	Where	Why
①	Single entry point (gateway, Facade)	Browser and store server → `api-gateway`	One place for routing, CORS, JWT check, rate limits and request IDs; service ports stay private.
②	Request/reply (synchronous REST)	parcel → pricing quote; merchant → create parcel; dispatch → change status; notification → contacts and webhook config	The caller needs the answer before it can reply. Calls go directly service-to-service with `X-Internal-Token`, 2 s / 5 s timeouts, and `503` on failure.

#	Pattern	Where	Why
③	Publish/subscribe through a broker	parcel-service publishes `ParcelStatusChanged`; four queues consume it	The status change should not wait for email, webhooks or reports. New consumers are added without touching the publisher, and a slow consumer cannot block a courier's update.
④	Webhook (outbound HTTP callback)	notification-service → merchant URL	Merchants learn about changes without polling. Each request is HMAC-signed and retried.
⑤	Server-Sent Events	parcel-service → tracking page, through the gateway	Updates flow one way only; SSE reconnects automatically and passes through the gateway and ingress, which is simpler than WebSockets (ADR-003).

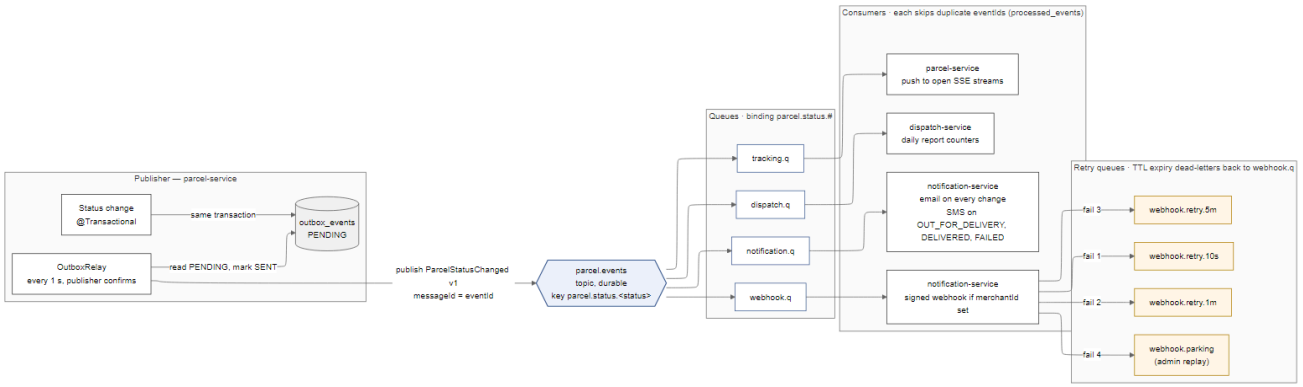


Figure 12. RabbitMQ topology: outbox publisher, topic exchange, four queues and the webhook retry queues

Broker (ADR-002). One durable topic exchange `parcel.events` with routing key `parcel.status.<status>`. Each consumer has its own queue bound to `parcel.status.#`, so a slow or failed consumer does not affect the others. Events carry IDs only; consumers fetch contact details at send time, so no personal data travels through the broker.

Reliable publishing (ADR-005). parcel-service writes the event to `outbox_events` in the same transaction as the status change; a relay publishes it with publisher confirms and marks it sent. If RabbitMQ is down, events wait in the table instead of being lost.

Idempotent consumers. Every consumer stores the `eventId` in its own `processed_events` table and skips duplicates, because the broker guarantees at-least-once delivery.

Webhook retry. A failed delivery goes to `webhook.retry.10s`, then `1m`, then `5m`; each TTL queue dead-letters back to `webhook.q`. After the fourth failure the message is parked, and a dispatcher or admin can replay it. Every attempt is recorded in `webhook_deliveries`.

6. Data design

6.1 Rules

- **Database per service.** One PostgreSQL server, six databases (`auth_db`, `pricing_db`, `parcel_db`, `dispatch_db`, `merchant_db`, `notification_db`), each with its own user.
- **No cross-database foreign keys.** A reference to another service's data is a plain `uuid` column (for example `parcels.customer_id`) with no constraint; the data is fetched through the owning service's API.
- **IDs are UUIDs**, timestamps are `timestamptz` in UTC, money is `numeric(10,2)` in LKR.
- **Flyway owns the schema.** `ddl-auto` is always `validate`; merged migrations are never edited.
- **Snapshot what must not change.** A parcel stores its fee breakdown, so the receipt stays the same if pricing rules change later.

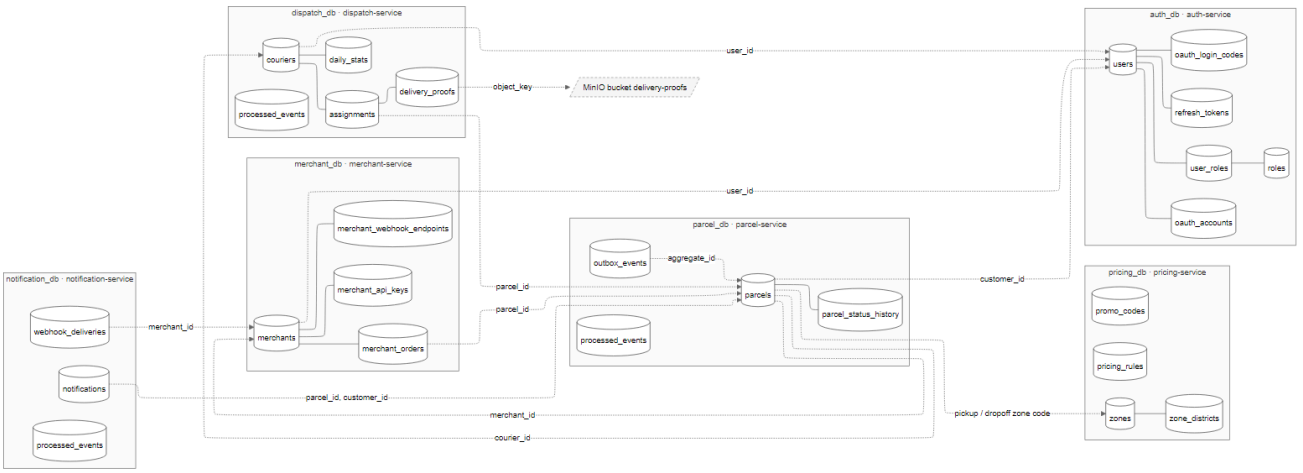


Figure 13. The six databases: foreign keys inside a database (solid) and logical references across databases (dashed)

6.2 Per-database ER diagrams

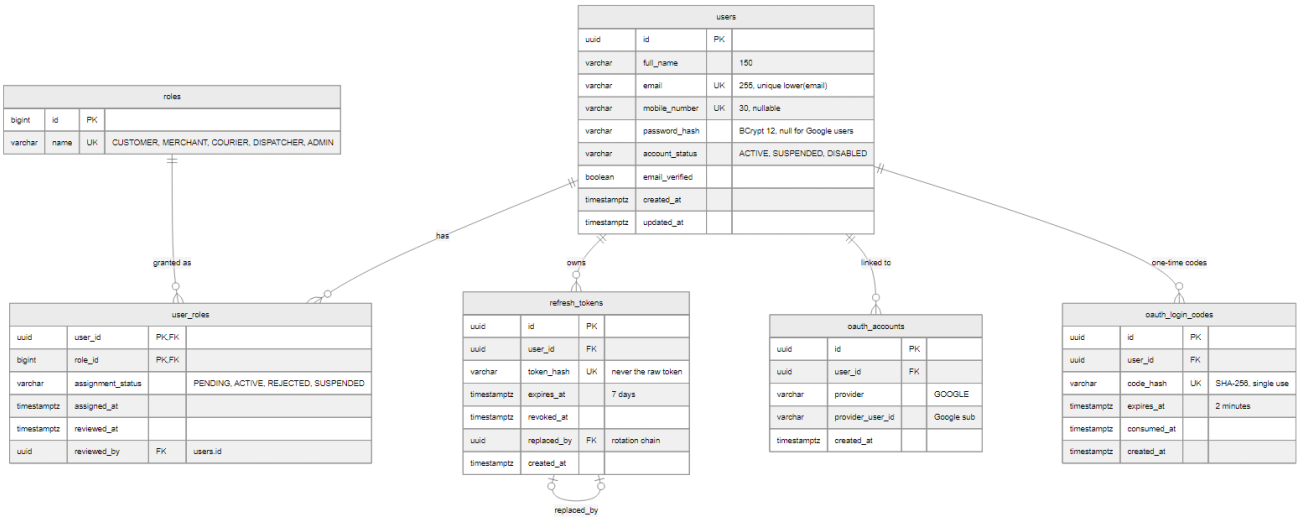


Figure 14. auth_db — identity only (users, roles, tokens, Google accounts, one-time login codes)

`auth_db` holds identity only (ADR-011). Refresh tokens and one-time Google login codes are stored as SHA-256 hashes; a refresh token points to its replacement, so a reused token can revoke the whole chain. Role 2's first migration also created `merchant_profiles` and `courier_profiles`; they are left out on purpose and will be removed (section 13).

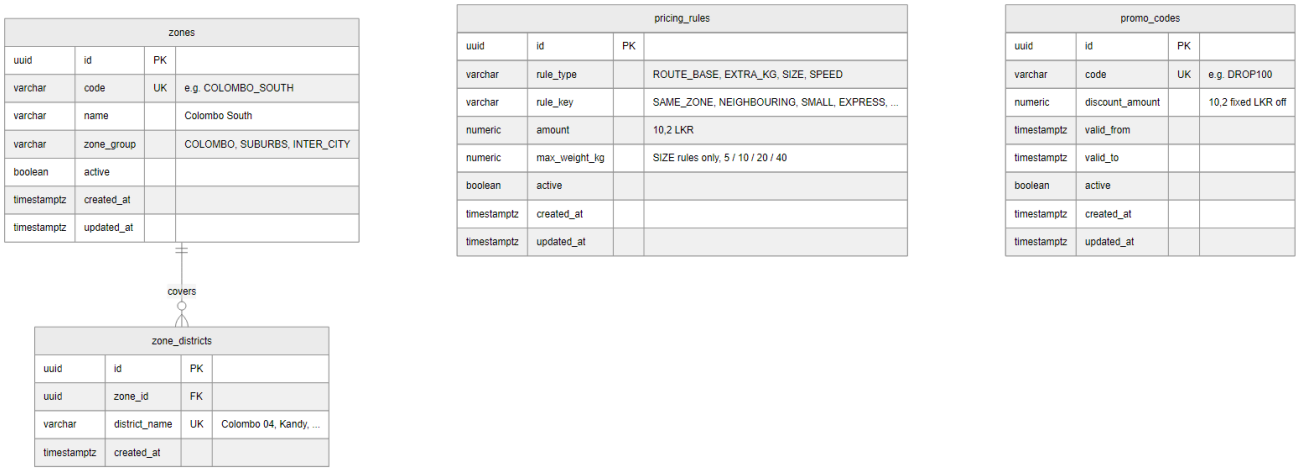


Figure 15. pricing_db (design)

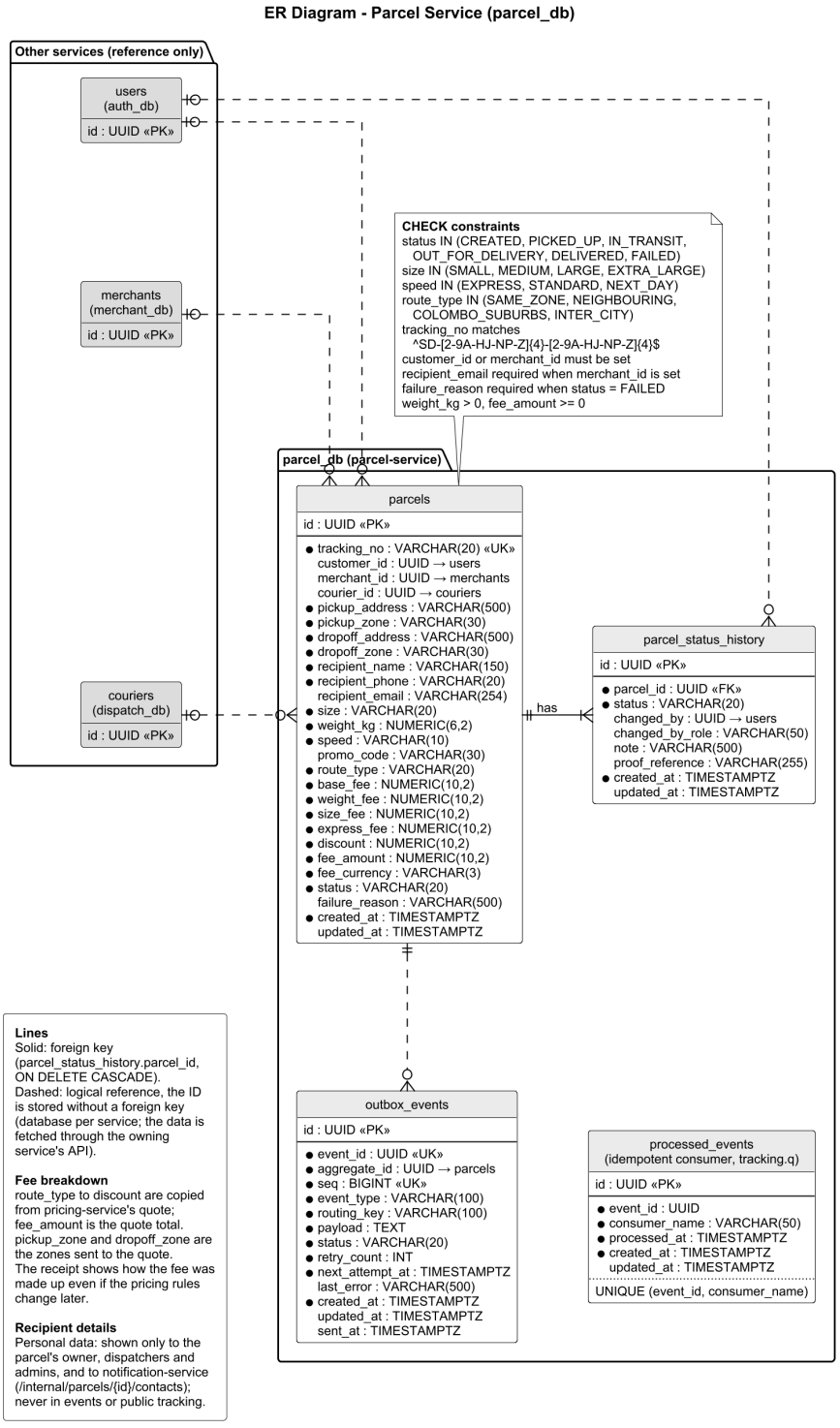


Figure 16. parcel_db (Sanduni, Role 3)

`parcel_db` keeps the status history with who, role, note and proof reference, and the outbox and processed-event tables for messaging. Check constraints enforce the status values, the tracking-number format and "failure reason only with `FAILED`".

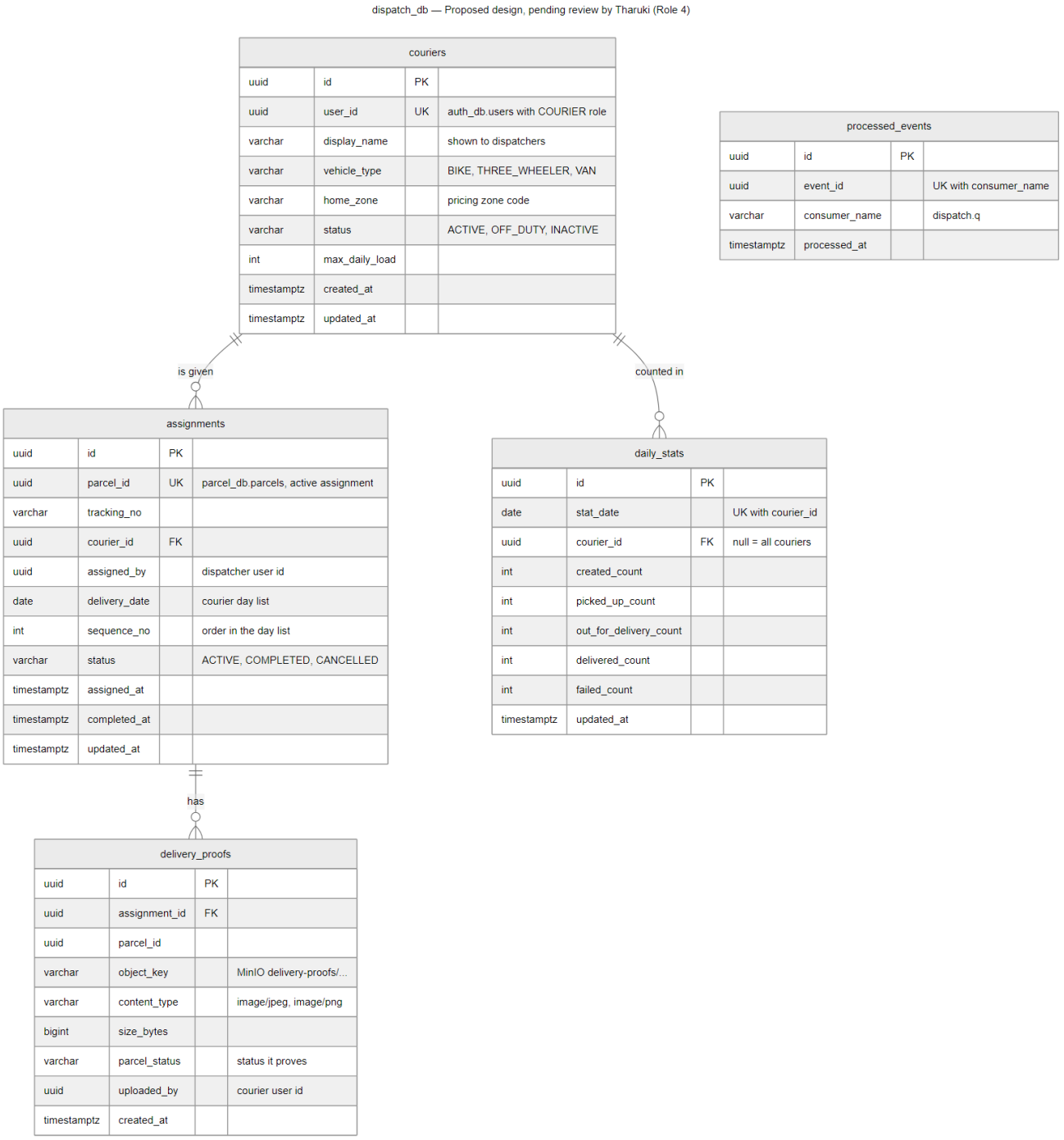


Figure 17. dispatch_db — proposed design, pending review by Tharuki (Role 4)

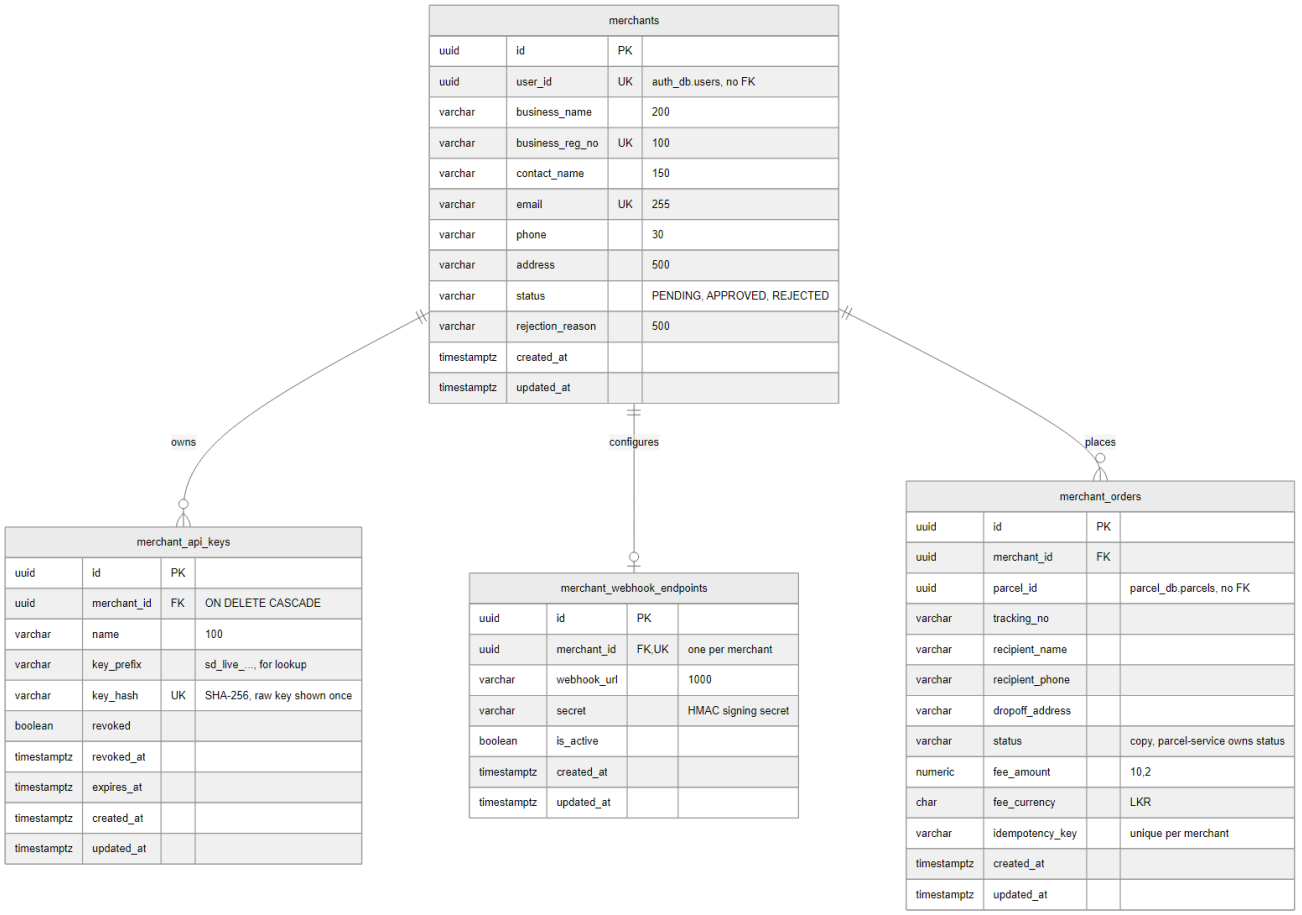


Figure 18. merchant_db (Sudari, Role 5)

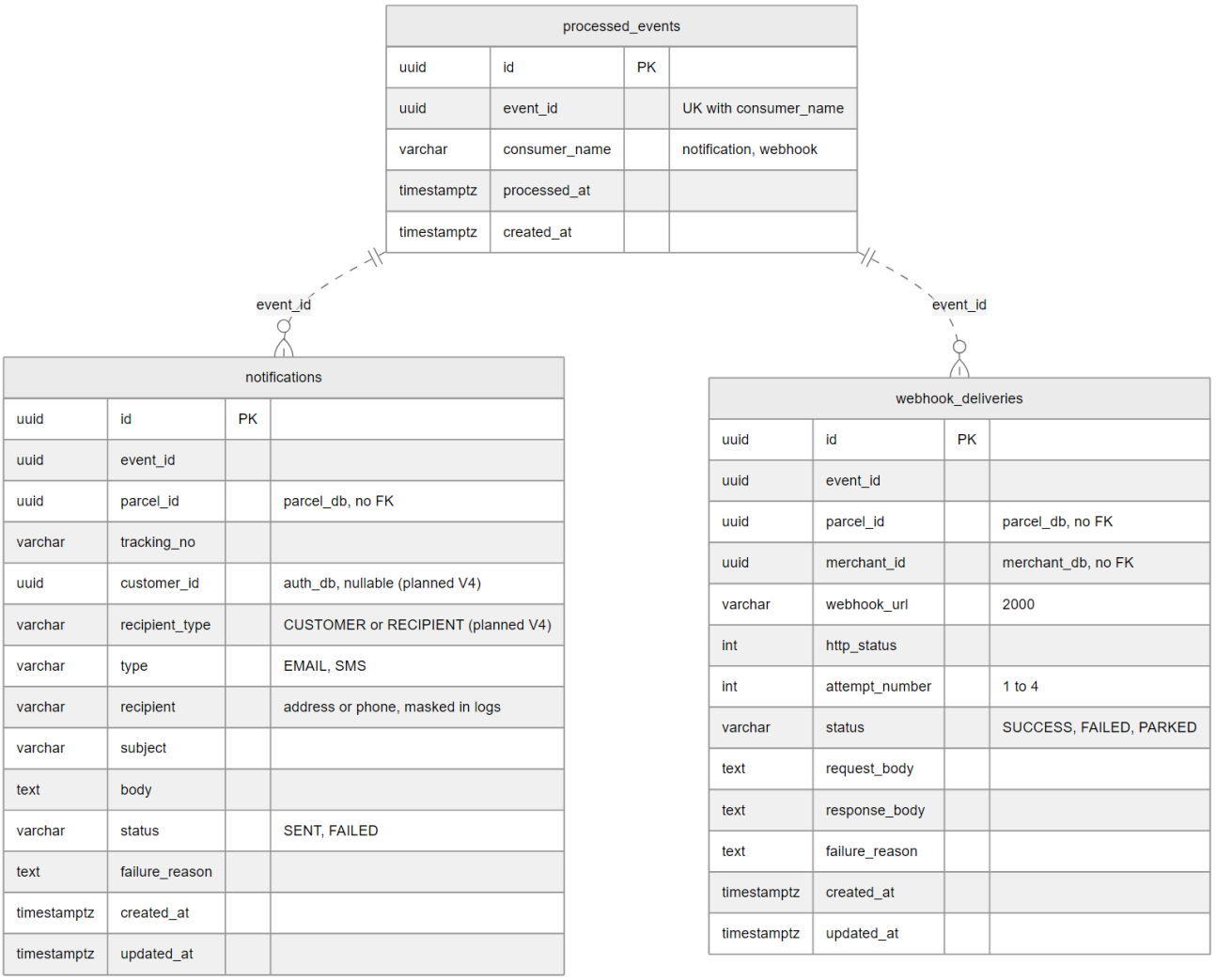


Figure 19. notification_db, with the planned nullable customer_id and recipient_type

`notification_db` records every email, SMS and webhook attempt. Merchant orders have no customer, so `notifications.customer_id` becomes nullable and `recipient_type` records whether the customer or the recipient was notified (planned migration, section 13).

7. Key flows

7.1 Sequence 1: Customer creates a delivery

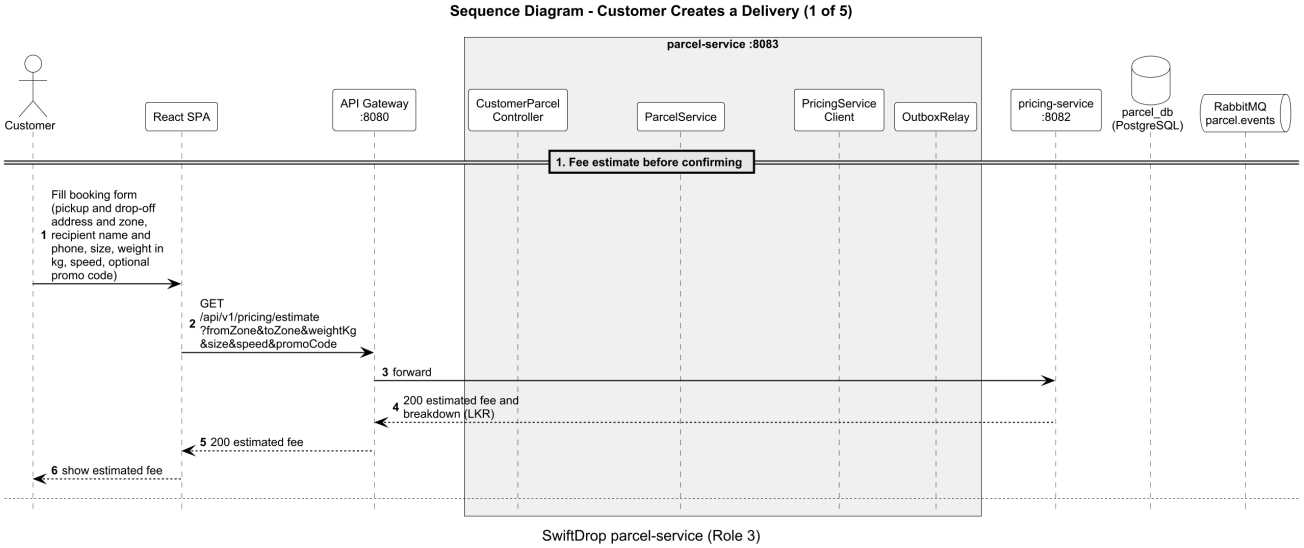


Figure 20a. Fee estimate (Sanduni, Role 3)

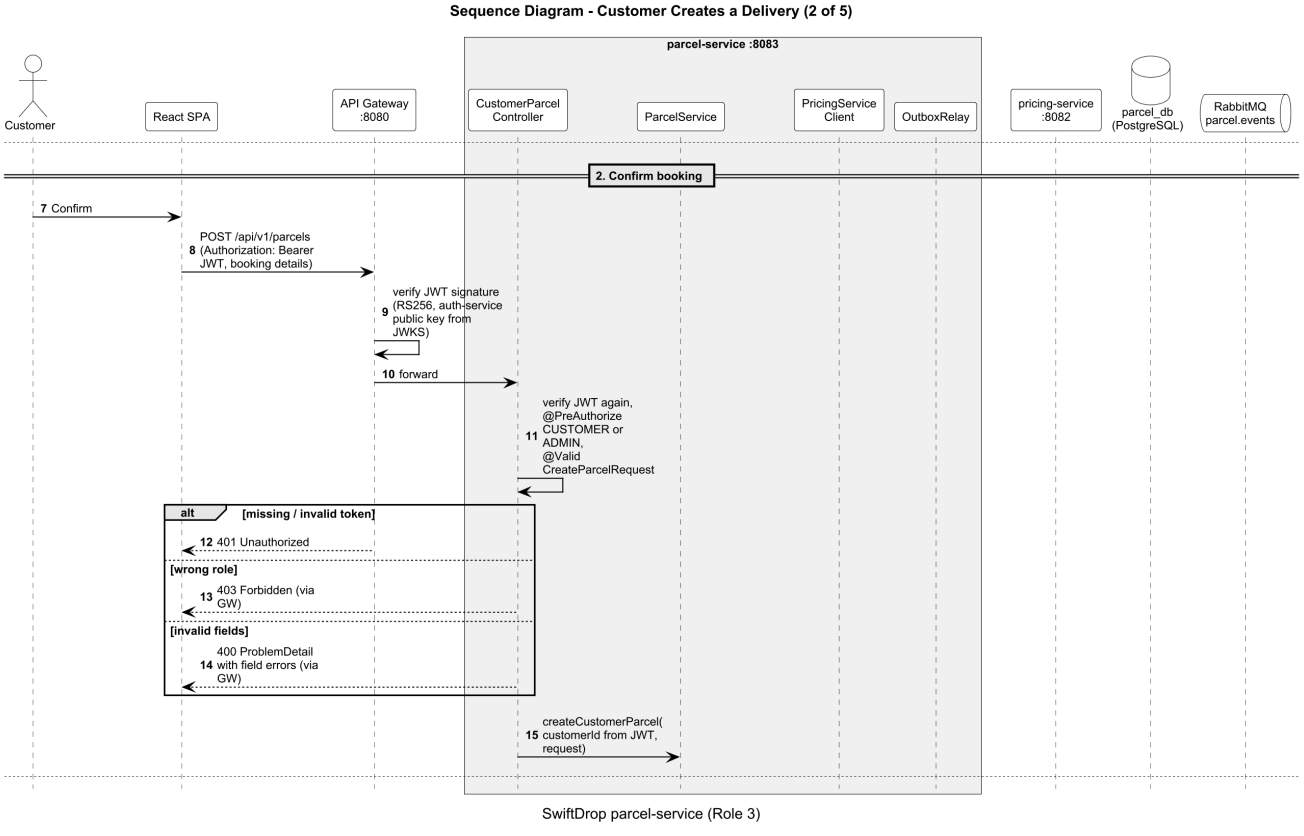


Figure 20b. Confirm booking: token, role and validation checks

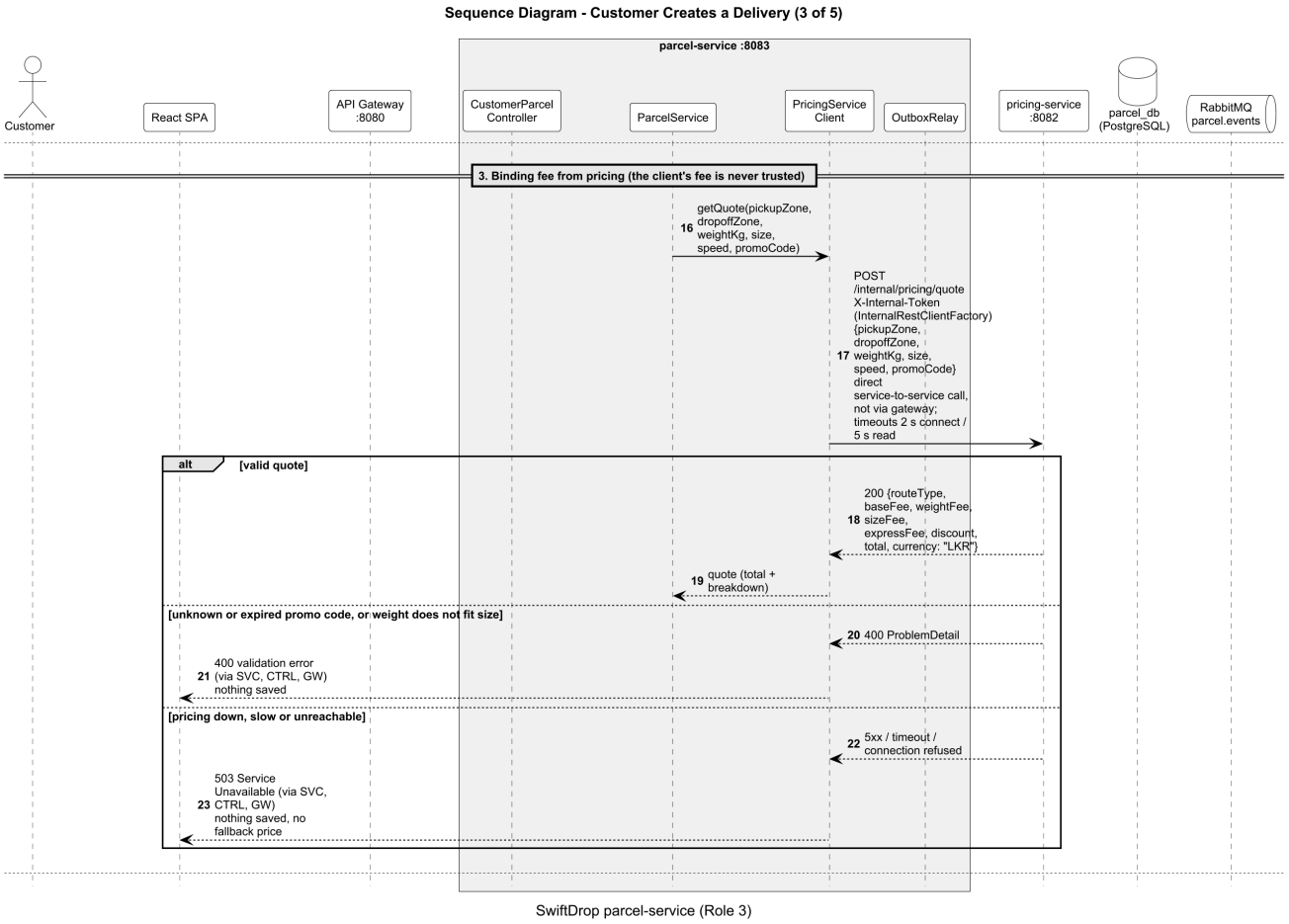
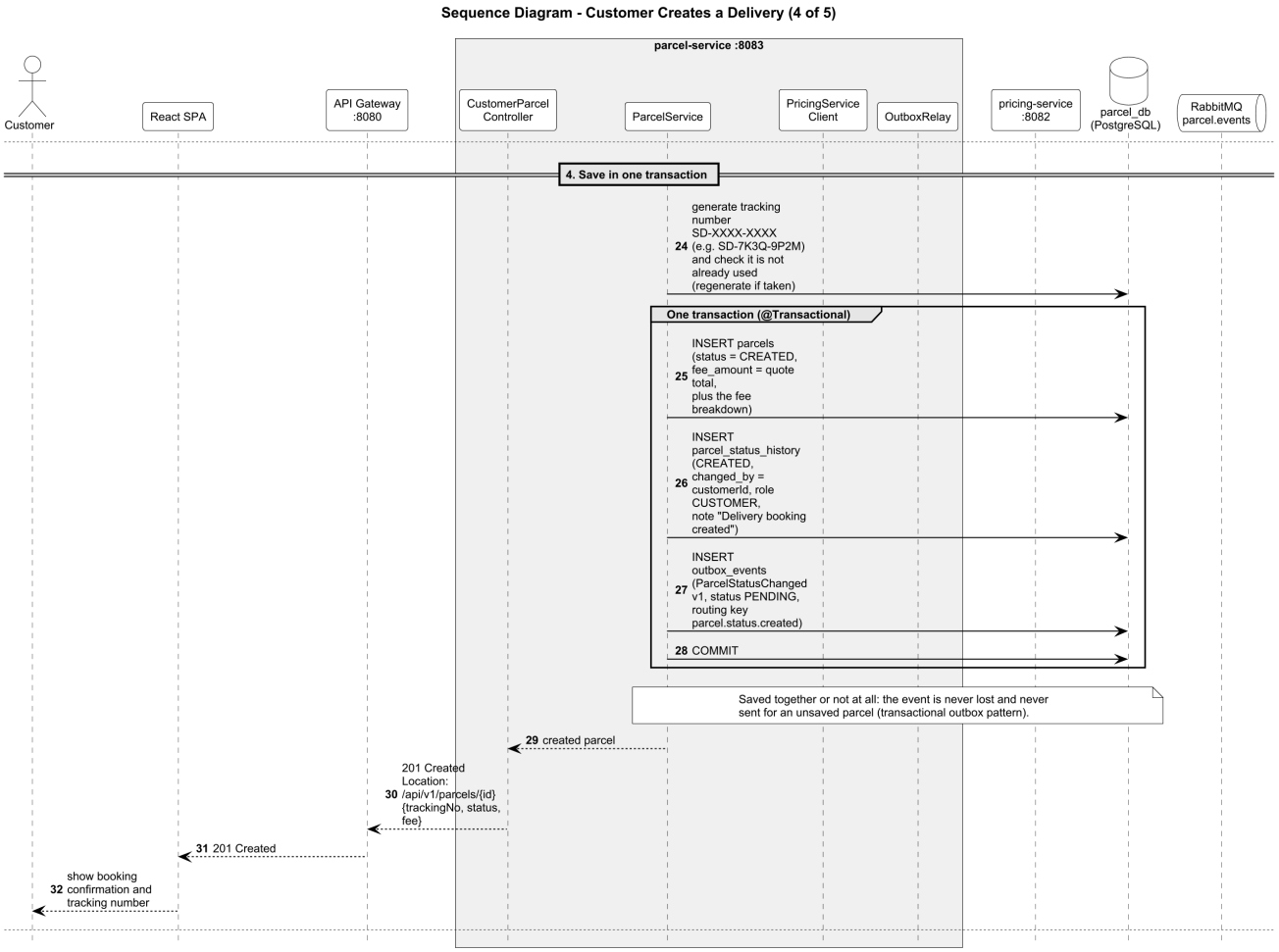
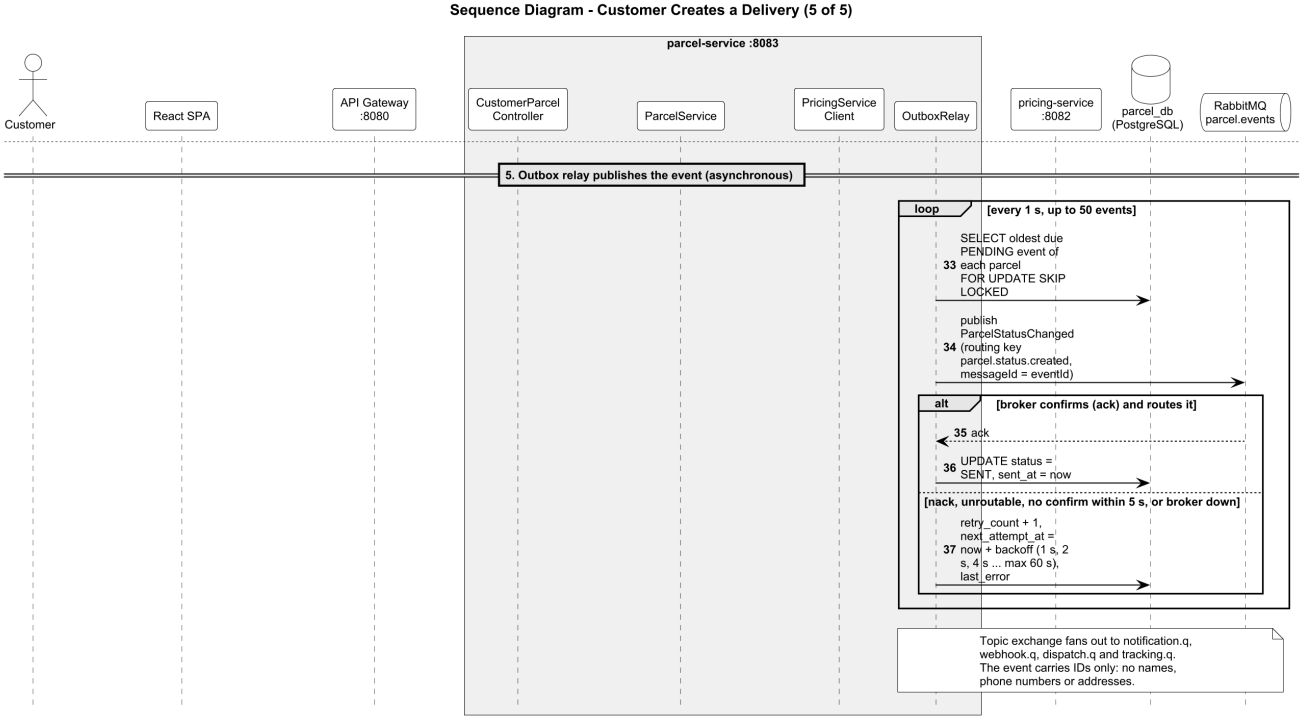


Figure 20c. Binding fee from pricing-service



SwiftDrop parcel-service (Role 3)

Figure 20d. Save parcel, history and outbox event in one transaction



SwiftDrop parcel-service (Role 3)

Figure 20e. Outbox relay publishes the event

The SPA shows an estimate first. On confirm, the gateway and parcel-service both verify the JWT; parcel-service then asks pricing-service for the binding quote over the internal API and saves the parcel, its first history row and the outbox event in one transaction. The relay publishes `parcel.status.created` about a second later. If pricing is down the booking fails with `503`; there is no fallback price.

7.2 Sequence 2: Merchant creates an order through the API

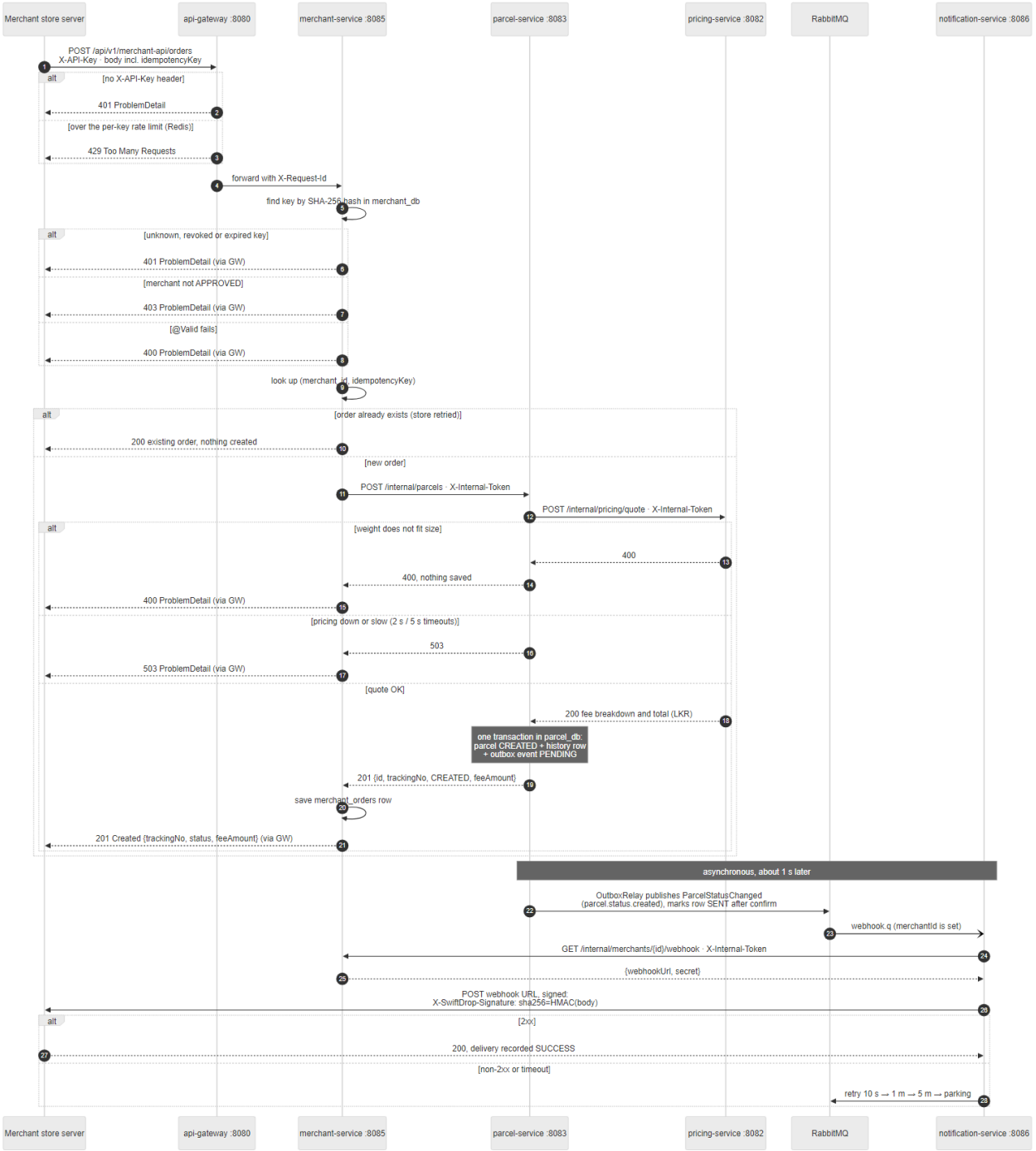


Figure 21. Merchant API order

The gateway rejects calls without `X-API-Key` (`401`) or over the per-key rate limit (`429`). merchant-service looks the key up by its SHA-256 hash and checks the merchant is approved. An idempotency key in the body makes store retries safe. The parcel is created through parcel-service's internal API with the same pricing rules as a customer booking, and the merchant later receives a signed webhook.

7.3 Sequence 3: Courier status update triggers notifications

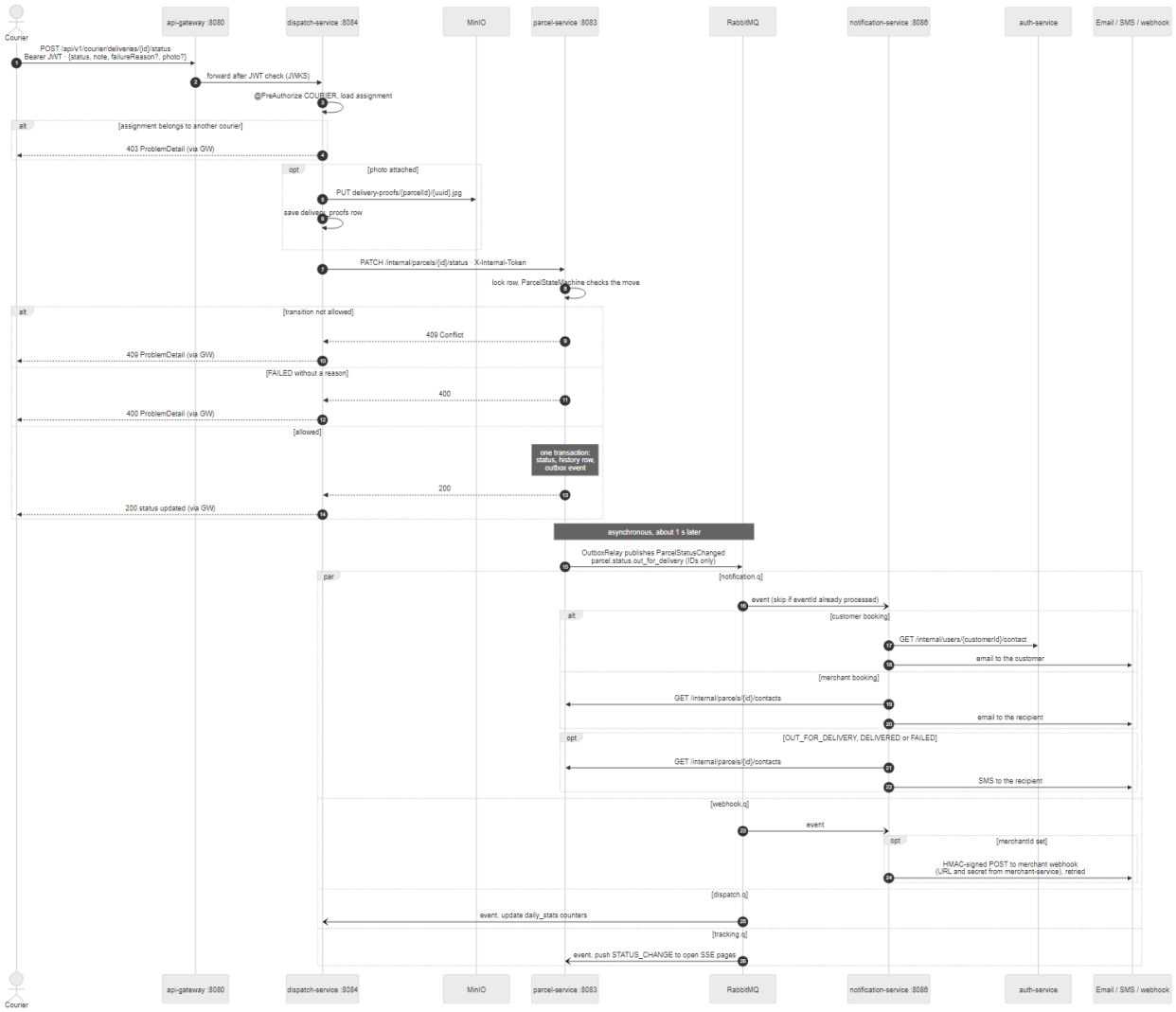


Figure 22. Courier status update and event fan-out

dispatch-service checks the courier owns the assignment and stores an optional photo in MinIO, then asks parcel-service to change the status; only parcel-service knows the transition rules. After the commit, one event fans out in parallel: email and SMS, the merchant webhook, report counters, and a push to open tracking pages.

7.4 Sequence 4: Login with Google (OAuth 2.0)

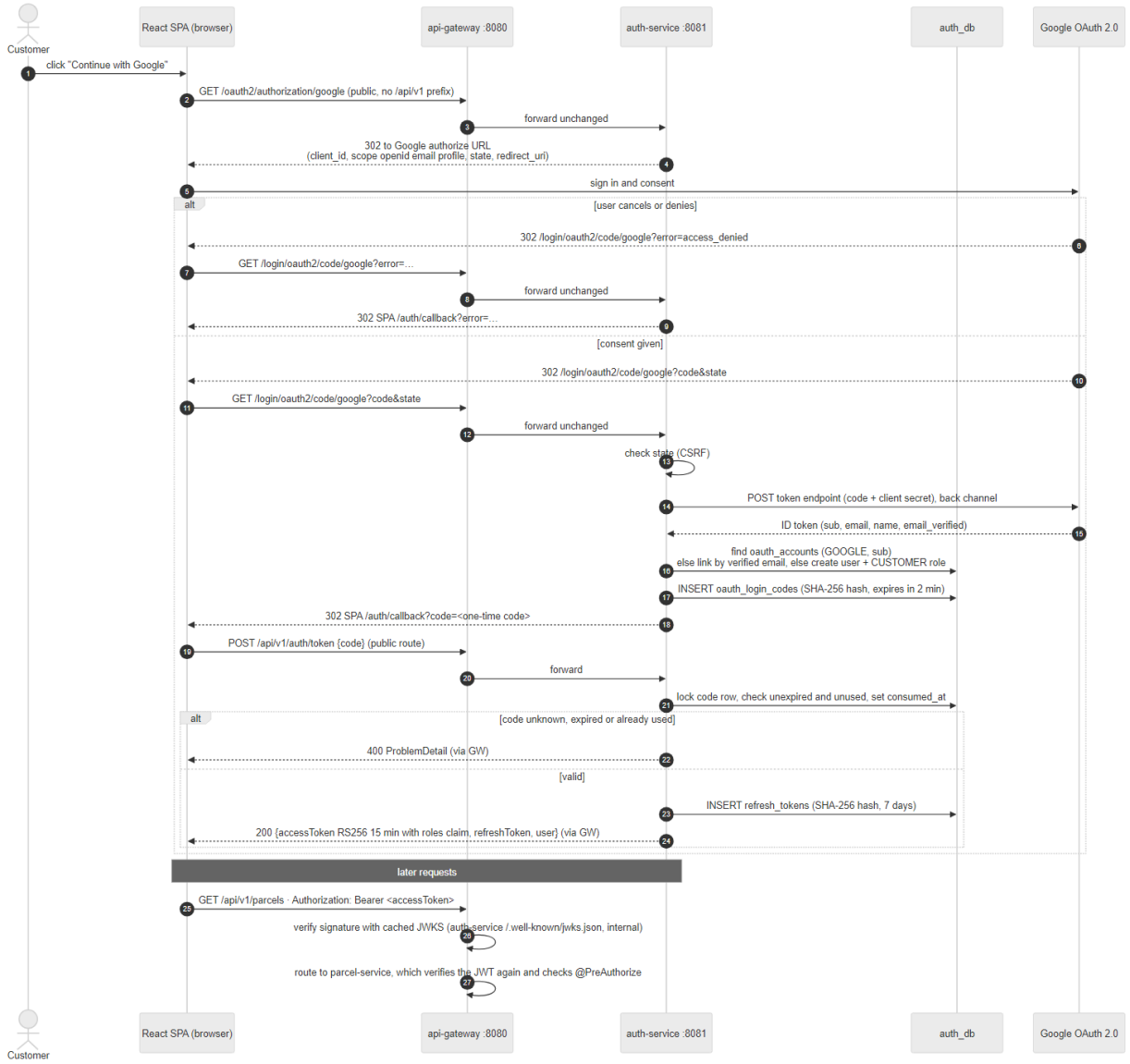


Figure 23. Google login with the Authorization Code flow

The gateway forwards Spring Security's default paths `/oauth2/**` and `/login/oauth2/**` to auth-service unchanged. auth-service exchanges Google's code on the back channel, finds, links or creates the user (new users get `CUSTOMER`), and redirects the SPA with a one-time code valid for two minutes. The SPA exchanges it for our own JWT and refresh token, so Google's tokens never reach the browser.

7.5 Flowcharts

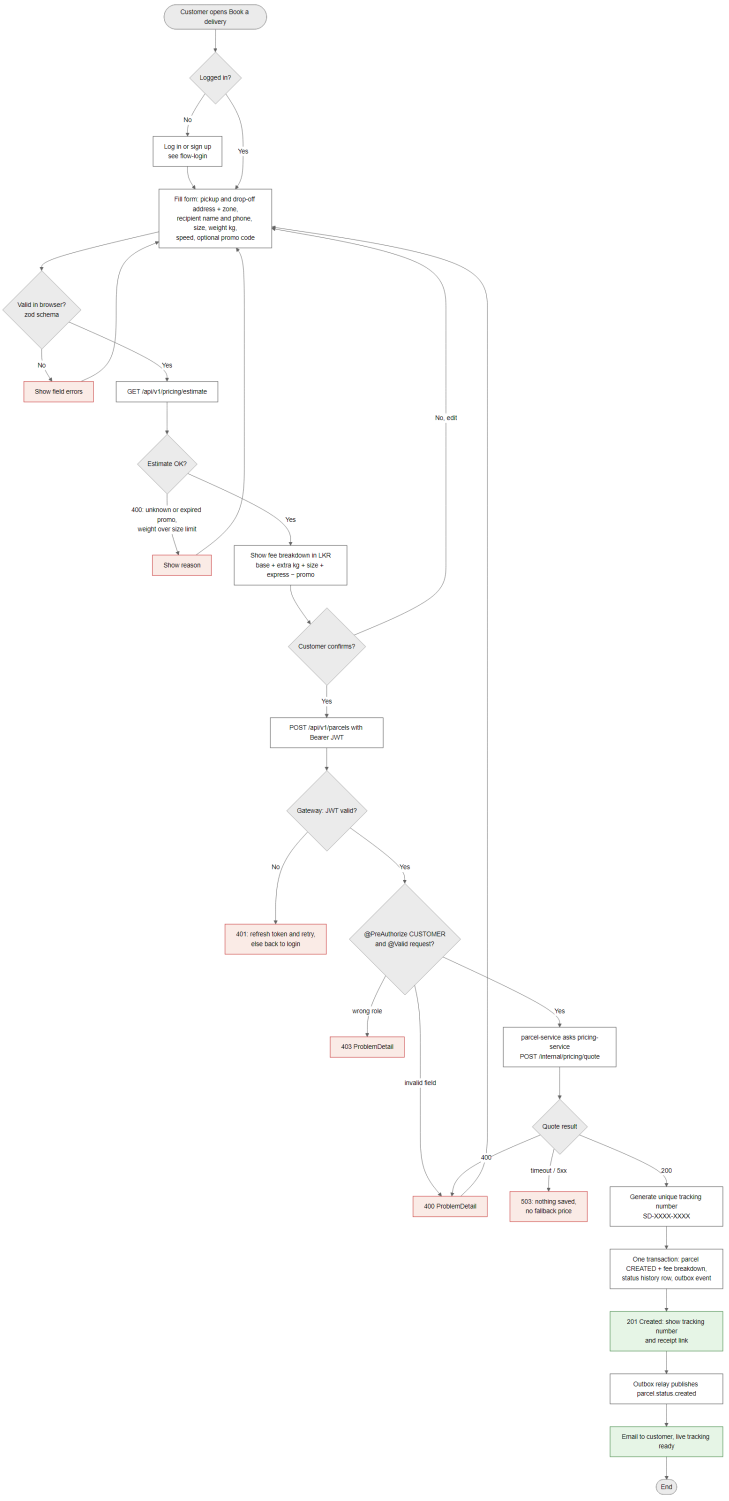


Figure 24. Booking flow

Booking (Figure 24). Errors are caught as early as possible: in the browser (zod), at the estimate, at the gateway and in the service. Every error sends the customer back to the form with a reason.

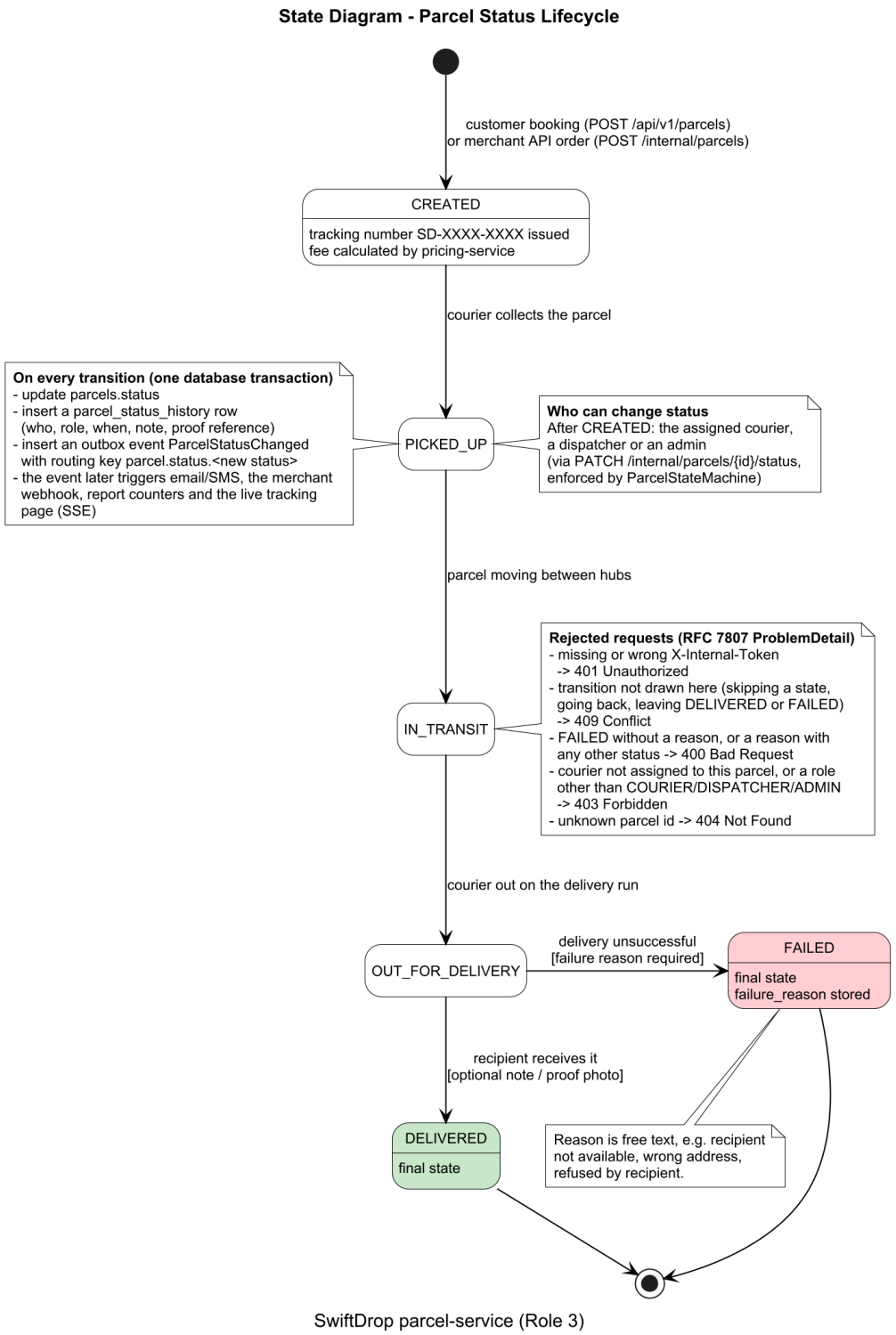


Figure 25. Parcel status lifecycle (Sanduni, Role 3)

Status lifecycle (Figure 25). Only forward moves are allowed, and `FAILED` is reachable only from `OUT_FOR_DELIVERY` with a reason. `DELIVERED` and `FAILED` are final. The detailed checks are in Sanduni's `activity-parcel-status-update` diagram.

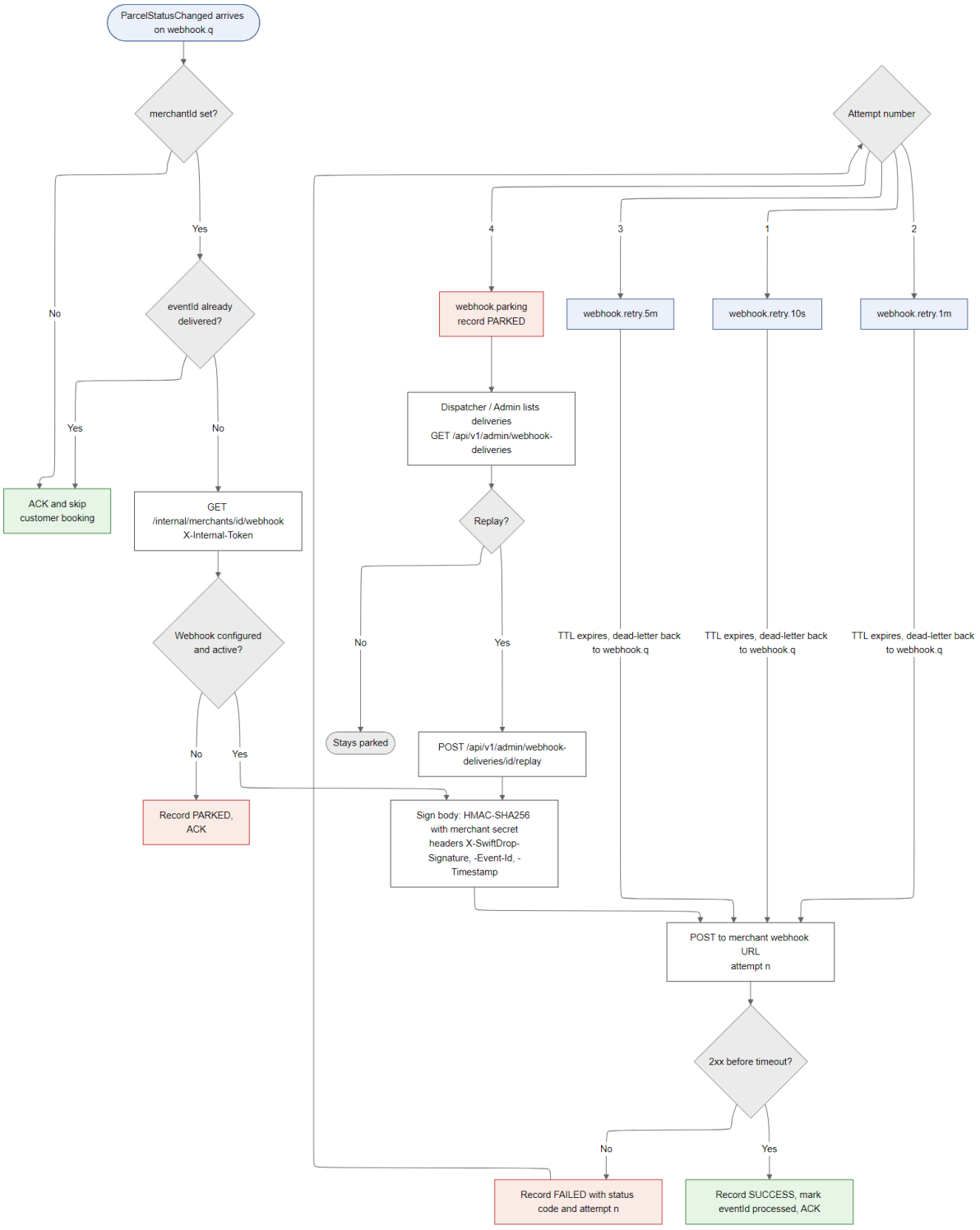


Figure 26. Webhook delivery, retry and replay

Webhook retry (Figure 26). Four attempts over about six minutes, each recorded; then the message is parked until a person replays it, so a broken merchant endpoint cannot block the queue.

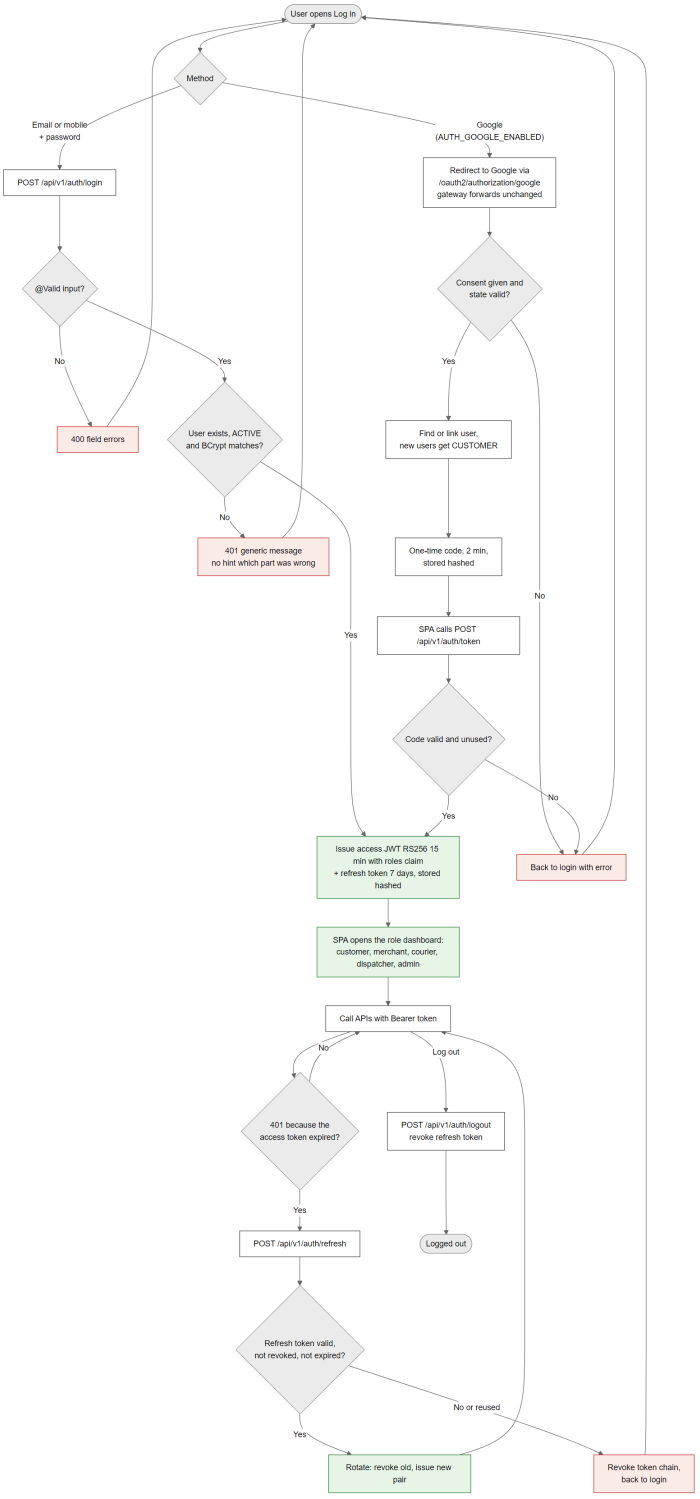


Figure 27. Login, Google login, token refresh and logout

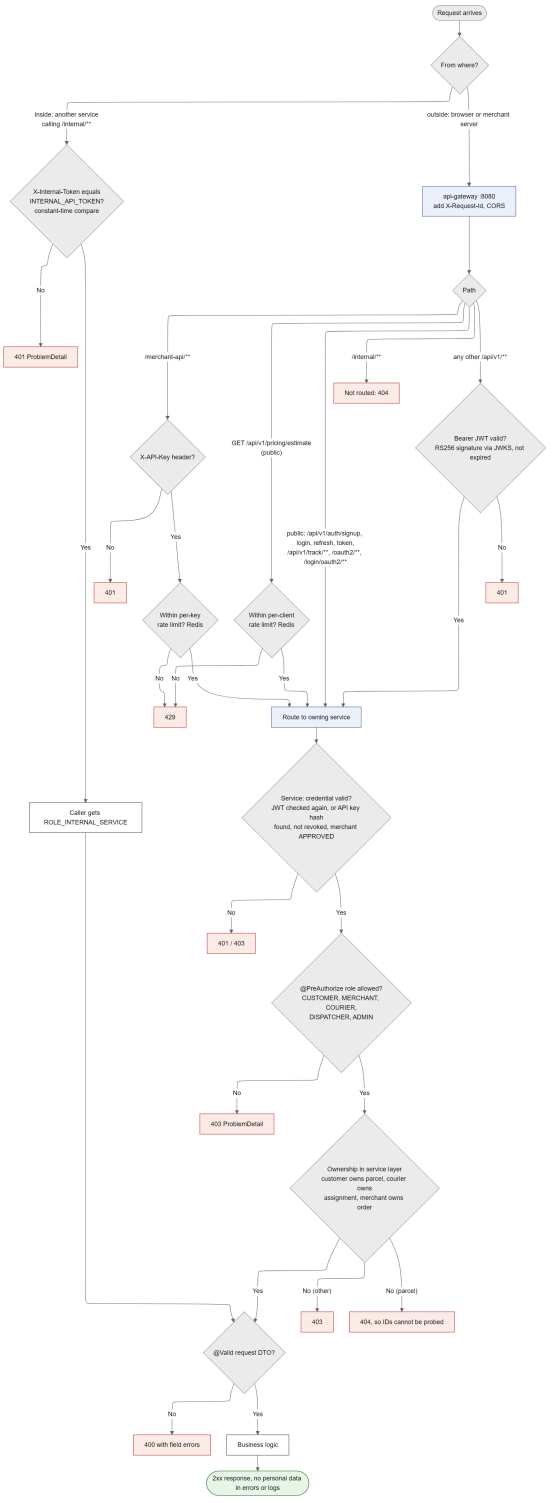


Figure 28. How every request is authenticated and authorized

Login and authorization (Figures 27–28). Wrong credentials get one generic `401`. Access tokens last 15 minutes; refresh tokens rotate on every use, and a reused one revokes the chain. Every request then passes the same checks in order: credential, role, ownership, validation.

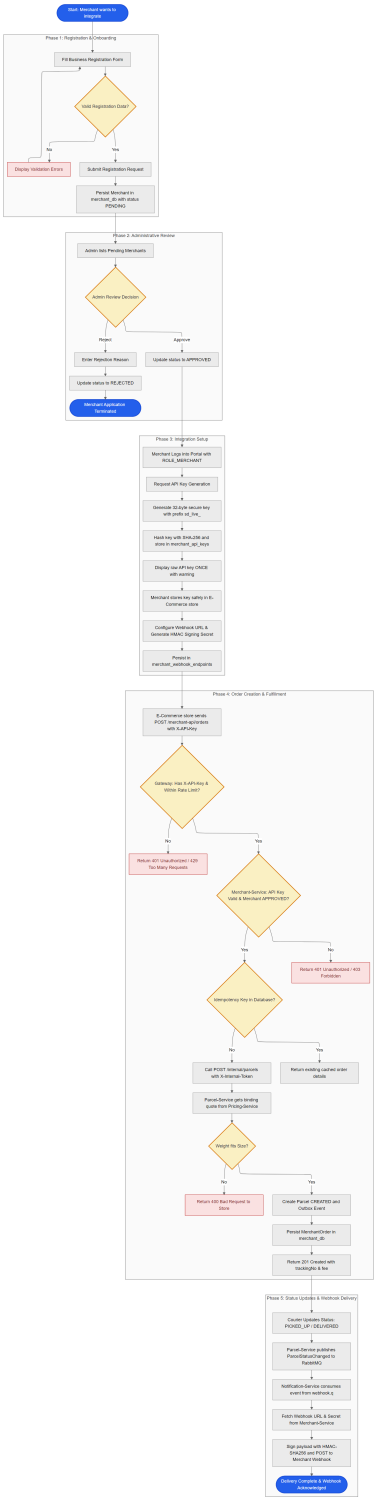


Figure 29. Merchant lifecycle — from branch feature/merchant, not yet merged (Sudari, Role 5)

Merchant lifecycle (Figure 29). From branch `feature/merchant`, not yet merged: onboarding, approval, API key and webhook setup, order creation and webhook delivery. The full-size image is `docs/report/img/activity-merchant-branch.png`.

8. Security design

Concern	Design
Passwords	BCrypt (strength 12), 12–72 characters. Google users have no password.
Access tokens	JWT signed with RS256, 15 minutes, `roles` claim. The private key comes from `JWT_PRIVATE_KEY_B64`.

Concern	Design
Token verification	auth-service publishes its public key at `/.well-known/jwks.json` (internal only). The gateway and every service verify tokens with it; there is no shared secret for user tokens.
Refresh tokens	7 days, stored as SHA-256 hashes, rotated on every use; logout revokes them.
Google login (customers)	OAuth 2.0 Authorization Code flow in auth-service, switched on with `AUTH_GOOGLE_ENABLED`. Verified email required; new users get `CUSTOMER`. The SPA receives a single-use, two-minute code, never Google's tokens.
Roles	`CUSTOMER`, `MERCHANT`, `COURIER`, `DISPATCHER`, `ADMIN`; `@PreAuthorize` on every endpoint, checked in the service and not only at the gateway.
Ownership	Checked in the service layer: customer ↔ parcel, courier ↔ assignment, merchant ↔ order. Unknown and "not yours" both return `404` for parcels.
Merchant API	`X-API-Key`, shown once, stored as SHA-256 hash with a lookup prefix, revocable, rate-limited per key at the gateway; merchant must be `APPROVED`.
Service-to-service	`/internal/**` requires `X-Internal-Token` equal to `INTERNAL_API_TOKEN`, compared in constant time (ADR-010). The gateway never routes `/internal/**`, and a user JWT does not open it. Services refuse to start without the token.
Webhooks	`X-SwiftDrop-Signature: sha256=<HMAC of body>`, plus event ID and timestamp so merchants can reject replays.
Personal data	No names, phones, addresses or tokens in logs, events or error responses; Logback masking in `common`; public tracking shows status only.
Errors	One handler in `common` returns RFC 7807 `ProblemDetail` with no stack traces, SQL or personal data.
Input	Bean Validation on every request DTO; zod in the browser.
Secrets	Environment variables (`.env` locally, Kubernetes Secrets in the cluster); nothing in Git.

9. Pricing design

```
fee = route base fee + extra weight + size surcharge + express surcharge − promo
```

The fee is calculated only in pricing-service. The public estimate (`GET /api/v1/pricing/estimate`) shows the expected fee; when a parcel is created, parcel-service asks for the binding quote (`POST /internal/pricing/quote`) and stores the total and its breakdown. A fee sent by a browser or a merchant is never used.

Component	Rule
Route base fee	`SAME_ZONE` 300 · `NEIGHBOURING` (same group) 350 · `COLOMBO_SUBURBS` 400 · `INTER_CITY` (either end inter-city) 650
Extra weight	First kg included; 40 per extra kg, rounded up; maximum 40 kg
Size	`SMALL` +0 (≤ 5 kg) · `MEDIUM` +100 (≤ 10 kg) · `LARGE` +250 (≤ 20 kg) · `EXTRA_LARGE` +500 (≤ 40 kg)
Speed	`EXPRESS` (3 hours) +100 · `STANDARD` (today by 8 PM) +0 · `NEXT_DAY` +0
Promo	Fixed amount off, e.g. `DROP100` = −100; the fee never goes below 0

Worked example. Colombo South → Colombo Central, 1.2 kg, `SMALL`, `EXPRESS`, promo `DROP100`:

Part	LKR
Base fee (`NEIGHBOURING`)	350
Extra weight (1.2 kg → 1 extra kg)	40
Size (`SMALL`)	0
Express	100
Fee before promo	**490**

Part	LKR
Promo `DROP100`	−100
Total	**390**

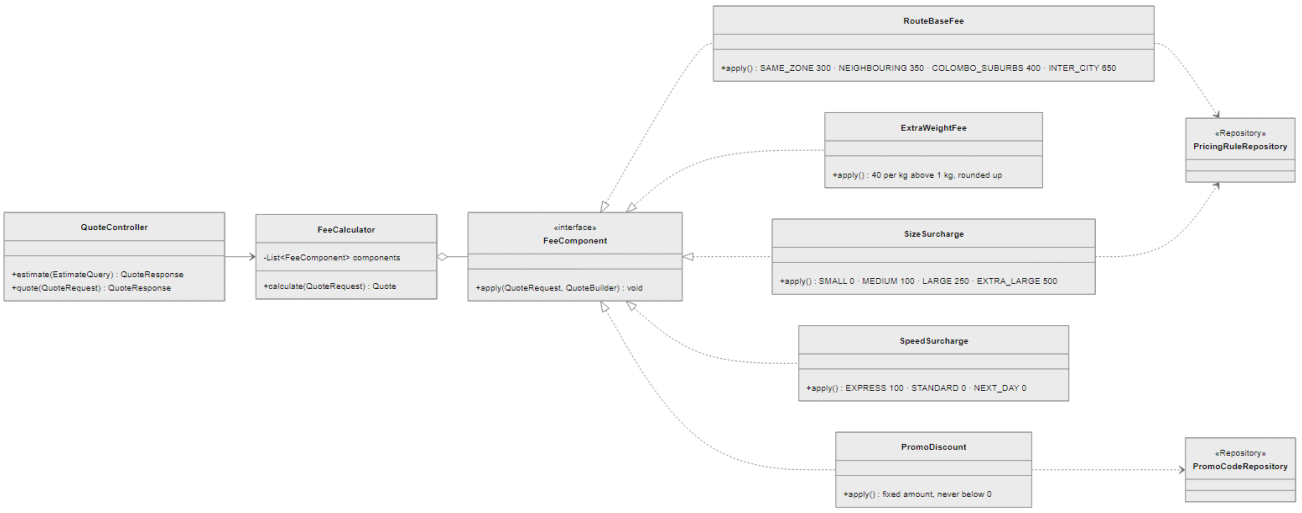


Figure 30. Pricing as a Strategy pattern (design)

Strategy pattern. Each component is a separate `FeeComponent` strategy. `FeeCalculator` applies them in order and builds the breakdown. A new surcharge, such as a fragile-item fee, is a new class, without changing the calculator (open/closed principle). The amounts live in `pricing_rules` and `promo_codes`, so admins change prices without a deployment.

10. Design patterns and principles

Pattern / principle	Where	Why
Single responsibility (SOLID)	One business capability per service; Controller → Service → Repository inside	Each class and service has one reason to change
Open/closed, dependency inversion (SOLID)	Pricing strategies; `NotificationChannel` interface; constructor injection only	Add behaviour without editing existing code; easy to mock in tests
DRY	`common` library: error handling, security chains, internal client, event DTOs, log masking	One implementation instead of seven
Repository	Spring Data JPA in every service	Persistence hidden behind interfaces; services stay testable
Strategy	Pricing fee components (section 9)	Each rule is replaceable and testable on its own
Observer / publish-subscribe	`ParcelStatusChanged` to four consumers	The publisher does not know its subscribers
State	`ParcelStateMachine` in parcel-service	Allowed transitions in one place; anything else returns `409`
Adapter	`EmailChannel`, `SmsChannel` behind `NotificationChannel`	Swap Mailpit for a real provider without touching the notification logic
Facade	API gateway	One simple entry point hides seven services
Transactional outbox	parcel-service `outbox_events` + relay	Status change and event are saved together; no lost or phantom events
Idempotent consumer	`processed_events` in each consumer	Safe under at-least-once delivery

Pattern / principle	Where	Why
Template method	Webhook retry handling	Same retry skeleton for every attempt, varying only the delay queue

11. Technology choices

Area	Choice	Justification
Language and framework	Java 21, Spring Boot 4.1	Mature ecosystem for REST, security, data and messaging; long-term-support Java; supported to mid-2027 (ADR-009)
Gateway	Spring Cloud Gateway 2025.1 (WebFlux)	Path routing, JWT resource server, Redis rate limiter and SSE pass-through in one component
Security	Spring Security 7, OAuth 2.0 client and resource server	Standard JWT/JWKS and Google login support without a separate identity server
Database	PostgreSQL 16 + Flyway	Reliable relational store with check constraints; versioned, reviewable migrations
Messaging	RabbitMQ (Spring AMQP)	Topic exchange, TTL and dead-letter queues make the retry design simple; management UI for the demo (ADR-002)
Cache / rate limits	Redis 7	Fast counters for the gateway's per-key and per-client limits
Object storage	MinIO (S3 API)	Proof photos outside the database; same API as AWS S3
Real-time	Server-Sent Events	One-way updates, automatic reconnect, works through the gateway (ADR-003)
Frontend	React 18, Vite, TypeScript (strict); planned Tailwind, React Router, TanStack Query, react-hook-form + zod	Fast builds, type safety, mobile-first styling, server-state caching
AI	Spring AI 2.x with an MCP server	Tools that call our own APIs with the user's token, so the assistant sees only what the user may see
Testing	JUnit 5, Mockito, MockMvc, Testcontainers, WireMock, Awaitility, JaCoCo; Vitest + React Testing Library	Unit, slice and real-infrastructure integration tests with a coverage gate
DevOps	Docker, Docker Compose, Helm, GitHub Actions, GHCR	One-command local runs; automated checks on every PR; Kubernetes bonus

12. UI design

The UI is designed in Figma: https://www.figma.com/design/xxZOVhob8n76HyUEJHJDoR/swiftdrop (page "SwiftDrop UI · v1", 55 screens). Every screen exists for desktop (1440 px) and mobile (390 px), except the courier screens, which are mobile-only because couriers work from their phones. A "Screen index — read me first" frame and a shared components frame sit next to the screens.

Area	Screens	Count
Shared	Sign in · Sign up · No access (403) · Not found (404), desktop and mobile	8
Customer	My deliveries · Book delivery · Fare & confirmation · Live tracking · Receipt · AI assistant · Profile, desktop and mobile	14
Merchant	Registration · Approval pending · Overview · Orders · Create order · Developer settings (API keys, webhook), desktop and mobile	12

Area	Screens	Count
Admin and operations	Platform overview · Users · Merchants · Zones & pricing · Dispatch · Couriers & workload · Reports · Webhook deliveries, desktop and mobile	16
Courier (mobile only)	Today's deliveries · Deliver parcel · Couldn't deliver	3
Public tracking	Track parcel, desktop and mobile	2
Total		**55**

The screens map onto the use cases in section 3. Error screens follow the API's status codes (`403`, `404`).

13. Current implementation status

State of `dev` at commit `2c99307` (31 October 2026), after Role 2's auth PR #26 and Role 3's recipient-contacts PR #28 were merged.

Module	Status	What works today
`common`	Done	ProblemDetail handler, JWT resource-server config, `X-Internal-Token` filter chain and `InternalRestClientFactory` (ADR-010), request IDs, PII log masking, event DTOs. 68 tests.
`auth-service`	Login complete	Sign-up (`POST /api/v1/auth/signup`, creates an `ACTIVE` customer), login with email or mobile number and password, refresh-token rotation, logout, Google login behind `AUTH_GOOGLE_ENABLED` with a two-minute single-use code exchanged at `POST /api/v1/auth/token`, JWKS at `/.well-known/jwks.json`, internal `GET /internal/users/{id}/contact` with the internal token. Admin user management is not built yet. 26 tests.
`frontend`	Login and dashboards	Sign-in and sign-up with a portal picker (Customer, Merchant, Courier, Staff), optional Google button, a role-based dashboard for each of the five roles, and merchant pages (registration, pending approval, API keys, webhook, dashboard, admin merchants). 10 tests.
`parcel-service`	Booking and internal API	Customer booking with the binding quote from pricing-service, merchant parcel creation over `/internal/parcels`, recipient contacts endpoint, transactional outbox and relay with retry back-off. 116 unit tests plus 4 Testcontainers integration test classes.
`merchant-service`	Mostly done	Registration, admin approve and reject, API keys (create, list, revoke), webhook settings, order API with idempotency, merchant order list and details, internal webhook lookup. 66 tests.
`notification-service`	Mostly done	Consumers for `notification.q` and `webhook.q` with idempotency, email and mock SMS routing per §6.3, HMAC-signed webhooks with the 10 s / 1 min / 5 min retry chain and parking, admin list and replay. 46 tests plus 2 integration test classes.
`api-gateway`	Started	Routes `/api/v1/auth/**` to auth-service only.
`pricing-service`, `dispatch-service`, `ai-assistant`	Not started	Application skeletons with one context test each.
`deploy`, CI	Compose and CI done	Docker Compose for all containers; PR pipeline (build, tests, JaCoCo, frontend checks) and release pipeline. The Helm chart is not written yet.

Known gaps and next steps

1. **Merchant route prefix.** The order controller is mapped to `/merchant-api/orders`, but the official path is `/api/v1/merchant-api/orders` (ADR-011). Through the gateway, which keeps the prefix, orders would return `404`. Role 5 to align the mapping.
2. **Auth profile tables.** `auth_db` still has `merchant_profiles` and `courier_profiles` from Role 2's V1 migration. By ADR-011 they belong in `merchant_db` and `dispatch_db`; a new auth migration should drop them.
3. **`notifications.customer_id`.** It is `NOT NULL`, but merchant orders have no customer. A V4 migration should make it nullable and add `recipient_type` (Role 6).

4. **Tokens in `sessionStorage`.** The SPA keeps the access and refresh tokens in `sessionStorage`, where any injected script could read them. Next step: access token in memory, refresh token in an `HttpOnly`, `Secure`, `SameSite` cookie.
5. **Demo accounts for non-customer roles.** Sign-up only creates customers, and admin user management is not built, so there is no supported way to get merchant, courier, dispatcher or admin accounts for the demo. Next step: a seed migration or start-up seeder for demo users (non-production only) and the `/admin/users` endpoints.
6. **Gateway routes beyond auth.** The gateway routes only `/api/v1/auth/**`. Still to build: routes for every service, `/oauth2/**` and `/login/oauth2/**` (the frontend currently calls auth-service directly on port 8081 for Google login), the JWT check, CORS, request IDs, rate limits and SSE pass-through.
7. **pricing-service not implemented yet.** parcel-service already calls `POST /internal/pricing/quote`, so bookings cannot succeed end-to-end until pricing exists (Role 1, design in section 9).
8. **Parcel status, history, receipt and SSE.** `PATCH /internal/parcels/{id}/status`, the list, details and receipt endpoints, the tracking page stream and the `tracking.q` consumer are designed but not yet on `dev` (Role 3).
9. **dispatch-service and ai-assistant.** Not started (Roles 4 and 9); the `dispatch_db` design in Figure 17 is pending Role 4's review.
10. **Frontend wiring.** Dashboards show static sample data and the merchant pages are not yet reachable from the app shell; React Router, TanStack Query and Tailwind are not added yet.
11. **Helm chart.** `deploy/helm` is empty.

14. Testing strategy and coverage

Level	Tools	What it proves
Unit	JUnit 5, Mockito	Service-layer rules: status transitions, pricing, notification routing, token rotation
Controller slice	MockMvc	Validation errors (`400`), authorization (`401`/`403`, at least one forbidden case per endpoint), ProblemDetail format
Integration	Testcontainers (PostgreSQL, RabbitMQ), WireMock, Awaitility	Flyway migrations, the outbox relay against a real broker, the notification flow, webhook retry timing
Contract	Shared event DTOs in `common`, contract test in parcel-service	Publisher and consumers agree on `ParcelStatusChanged` v1
Frontend	Vitest, React Testing Library	Login, portal selection and role dashboards
End-to-end (planned)	Docker Compose demo script	Booking → assignment → status update → email, webhook and live tracking

Test names describe behaviour, for example `courierCannotViewAnotherCouriersDelivery()`. CI fails a pull request if a module's line coverage drops below 70%.

Current results (31 October 2026, unit tests, `./mvnw test jacoco:report` on `dev` at `2c99307`; all tests pass):

Module	Tests	Line coverage	Branch coverage
`common`	68	93.2% (317/340)	76.6%
`parcel-service`	116	82.9% (447/539)	73.9%
`auth-service`	26	81.8% (266/325)	62.5%
`notification-service`	46	81.8% (368/450)	78.6%
`merchant-service`	66	77.4% (514/664)	57.9%
`api-gateway`, `pricing-service`, `dispatch-service`, `ai-assistant`	1 each	skeletons	—
`frontend` (Vitest)	10	not measured yet	—

The six Testcontainers integration test classes (four in parcel-service, two in notification-service) need Docker and run in CI's `verify` phase. They were not run for these numbers, so the coverage above is from unit and slice tests only.

15. Team contributions

From `git log` on `dev` (merge commits excluded) and the open branches, 31 October 2026.

Member	Role	Modules (commits on `dev`)	Diagrams authored
N.L. Sachintha Dilshan	1 · Team lead	`common`, `docs`, service skeletons, CI set-up, `pricing-service` and `api-gateway` (started) · 39 commits	All diagrams in `usecase/`, `er/` (except parcel), `architecture/`, `integration/`, `sequence/`, `flowchart/`, corrections credited to their original authors; prepared with Claude Code (AI coding assistant), as the commit trailers record
Thulani Jayathilake	2 · Auth	`auth-service`, frontend login and dashboards · 3 commits (PR #26)	Auth use case, auth architecture, password login, refresh token, Google login and internal contact lookup (the last two recreated)
Sanduni Liyanage	3 · Parcel	`parcel-service` · 10 commits	Create delivery, live tracking, history and receipt, internal parcel API, parcel_db ER, status lifecycle, status update activity, parcel components
Tharuki	4 · Dispatch	No commits yet	None yet; reviewer of the proposed `dispatch_db` design and the courier part of the status sequence
Sudari Ratnayake	5 · Merchant	`merchant-service`, frontend merchant pages, Postman collection · 5 commits	Merchant use case, merchant_db ER, merchant order sequence, merchant lifecycle (branch `feature/merchant`)
Thisaru Ramanayaka	6 · Notifications	`notification-service`, RabbitMQ topology · 22 commits	Notification use case, architecture, email/SMS sequence, webhook retry and replay sequences, courier status sequence
Supuni	7 · Frontend	Customer use case diagrams and descriptions · 1 commit	UC-00 to UC-04 customer use cases (UC-02 and UC-03 recreated)
Kevinda	8 · DevOps	`deploy/compose`, Dockerfiles, GitHub Actions · 3 commits	—
Devmith	9 · AI and testing	No commits yet	—

The 55 Figma screens are listed in section 12.

16. Appendix

16.1 Architecture decisions

ADR	Decision	Status
001	Microservices behind an API gateway (modular monolith considered)	Accepted
002	RabbitMQ for status events, retry with TTL and dead-letter queues	Accepted
003	Server-Sent Events for live tracking	Proposed
004	API keys for merchant API access	Proposed
005	Transactional outbox in parcel-service	Proposed
006	Custom auth-service instead of Keycloak	Proposed
007	Spring Boot 4.0 line	Superseded by 009
008	Database per service, migrations reviewed by the team lead	Accepted
009	Spring Boot 4.1 line	Accepted
010	Shared internal token for service-to-service calls	Accepted
011	Identity-only `auth_db`, public fee estimate, OAuth paths without prefix	Accepted

Full text: `docs/decisions.md`.

16.2 Glossary

Term	Meaning
API gateway	Single entry point that routes external requests to services and checks credentials
Courier	The person who picks up and delivers parcels (role `COURIER`)
DLX / dead-letter	RabbitMQ mechanism that moves an expired or rejected message to another exchange; used for webhook retry
Flyway	Tool that applies versioned SQL migrations at start-up
HMAC	Keyed hash used to sign webhook bodies so merchants can verify the sender
Idempotent	Processing the same message or request twice has the same effect as once
JWKS	JSON Web Key Set: the public keys used to verify JWT signatures
JWT	JSON Web Token: a signed token carrying the user ID and roles
MCP	Model Context Protocol: lets the AI assistant call our APIs as tools
Outbox	Table where an event is saved in the same transaction as the data change, then published by a relay
ProblemDetail	RFC 7807 JSON error format used by every service
SSE	Server-Sent Events: a one-way stream from server to browser over HTTP
Tracking number	Public parcel reference in the form `SD-XXXX-XXXX`
Webhook	HTTP callback from SwiftDrop to a merchant's server when a parcel changes